🔐 KeyPrint Guard & Arena: 타이핑 생체 지문(Keystroke Dynamics) 임베딩 기반 실시간 해커 침입 차단(1:1) 및 타자연습 인물 자동 분류(1:N) 시스템

> **"비밀번호가 유출되어도 해커는 당신의 손가락 근육 리듬을 복제하기 어렵고, 똑같은 타자연습 문장을 쳐도 AI는 누가 타이핑했는지 식별합니다."** 키보드의 미세 누름 지속 시간(`Hold`), 키 간 전환 지연(`DD`), 비행 시간(`UD`) 및 연타 롤오버(`UD < 0`) 파형으로부터 **64차원 초구면 생체 지문 임베딩(64-D Hyperspherical Biometric Embedding)**을 추출하여, 별도 추론 서버 없이 브라우저 내부(ONNX Runtime Web WASM)에서 **약 1.5~5ms**(Python 로컬 순방향 추론 **0.4~0.9ms**) 만에 **① 유출 암호 해커 침입 차단(1:1 검증)**과 **② 타자연습 다중 인물 자동 식별(1:N 분류)**을 수행하는 End-to-End 머신러닝 행동 생체인식 시스템입니다. *(단, 선택 기능인 다중 기기 방 동기화 사용 시 등록 프로필과 시연용 암호 문자열이 공개 MQTT 브로커로 공유됩니다 — §6.2, §7.2).*

> **v2 업데이트 (`kp-v4` 전처리·누수 보정 및 듀얼 인코더 확장)**: ① 어떤 키에서 어떤 키로 이동했는지를 담은 **키 위치 7채널 + 누름 시간 유효 플래그**를 더한 17채널 문장 인코더로 웹 동일 재현(`68자` 캡처) 기준 미학습 18인 식별(다른 날·다른 글) Top-1을 **`61.36%` → `77.28%`** , ② 백스페이스·Shift·멈춤 습관을 점수화한 **행동 프로필(`β = 0.15`)**을 더해 **`81.30%`** (같은 날 `85.87%` , 다른 날 Top-3 `96.45%`)까지 끌어올렸고, ③ 모바일 소프트 키보드에서 프로필 등록이 막히던 문제를 `input` 차분 복원 녹화기로 해결했습니다(근거: §5.5). ④ 이어서 미학습 시험 인원 18명의 디그래프 타이밍을 등록·질의 표본 비중첩(`occ_id` 분리) 조건으로 재조합한 **합성 문자열 60종(생성 입력 수: 본인 `5,958` 회 / 사칭 `31,196` 회)**으로 검증하여, 비밀번호는 **9채널+17채널 듀얼 인코더 표준화 결합 + 고신뢰 등록 리듬 자동 갱신(`z ≥ τ + 1.5`)**으로, 타자연습은 **등록 5문장 + 판별 2문장**으로 고도화했습니다(10인 갤러리 외부인 오통과율 `6.6%` → `1.3%` , 근거: §5.6). ⑤ 신규 비밀번호는 **12자 이상**만 허용하고 길이별 보정 임계값과 대문자 입력 습관(`Shift` vs `CapsLock`)을 결합해, 12자 키보드 비밀번호 사칭 통과율을 `40.2%` → `19.1%` (첫 글자 대문자 포함 시 `18.2%` → `11.0%`)까지 낮췄습니다(근거: §5.7).

---

0. 프로젝트 정보

항목	내용
과목명	기계학습 프로그래밍 (기말 포트폴리오 프로젝트)
학번 / 이름	`201935607` / **김대현**
프로젝트명	**KeyPrint Guard & Arena**: 키스트로크 다이나믹스 생체 임베딩 기반 실시간 해커 침입 차단(1:1) 및 타자연습 인물 자동 분류(1:N) 시스템
GitHub 저장소	https://github.com/qwertypotter6873/keyprint-guard
배포 데모 URL	https://qwertypotter6873.github.io/keyprint-guard/
데이터 유형	시계열·정형 행동 생체 신호 (`Tabular & Temporal Behavioral Biometrics`)
학습 방식	**직접 학습 (`Scratch + Deep Metric Learning`)** — 다중 코퍼스·가변 길이 시퀀스 통합 메트릭 러닝
비교 및 제안 모델	① `ScaledManhattanDetector` (통계 베이스라인) · ② `Vanilla1DCNN` 및 통제 변형(`+증강` , `+AS-Norm`) · ③ **`KeyPrintNet` 9채널 / 17채널** (제안 모델, 최종 채택)

항목	내용
주요 기술 스택	`Python 3.12`, `PyTorch`, `Scikit-Learn`, `SciPy`, `ONNX (Opset 17)`, `ONNX Runtime Web (WASM SIMD)`, `Vite`, `Vanilla ES2022 JS`, `GitHub Actions / Pages`

---

1. 문제 정의

1.1 해결하려는 실제 문제 배경 (4W 및 기존 방식 대비 차별점)

1. **유출 암호 기반 계정 탈취(Credential Stuffing & Phishing)의 구조적 취약성**: 기존 텍스트 비밀번호 인증은 문자열 일치(`typed == stored`)만을 검사하는 이진 논리에 의존합니다. 피싱, 다크웹 DB 유출, 어깨너머 훔쳐보기(Shoulder Surfing)로 비밀번호 문자열이 노출되는 순간 인간 사칭자나 자동화 매크로(`AutoHotkey`, `Selenium` 등)의 침입을 막지 못하며, OTP·물리 보안키는 추가 기기 조작 마찰을 유발합니다.
2. **공유 텍스트·타자연습 환경에서의 무자각 행위자 식별("Who Is Typing?")**: 여러 사용자가 동일한 연습 문장이나 표준 명령어를 타이핑할 때 결과 문자열은 동일하지만, 손가락 길이·도약 습관·연타 롤오버(`UD < 0`) 비율이 고유한 신경근육 기억(Neuromuscular Memory)을 형성합니다. 이를 초구면 임베딩 공간으로 사상하면 짧은 타자연습 등록만으로 이후 자유 문장 작성자를 실시간 분류할 수 있습니다.

구분 (4W)	정의 및 핵심 내용
Who (대상 사용자)	① 비밀번호 유출·공유 위험에 노출된 웹 서비스 사용자 및 사내 시스템 관리자, ② 온라인 타자연습·코딩 실습·LMS 시험에서 별도 장비 없이 작성자 본인 여부를 확인하려는 교육·평가 플랫폼
What (해결 과업)	키 입력·해제 타임스탬프만으로 `64-D` 생체 임베딩을 추출해 **유출 암호 침입 차단(`1:1 Open-Set Verification`)**과 **자유 문장 작성자 식별(`1:N Multi-User Identification`)**을 동시 수행
When (작동 시점)	로그인 비밀번호 입력 직후(`Enter`) 및 한글·영문 짧은글 타자연습 문장 입력 시점(오타 수정용 `Backspace` 자유 허용)
Why (필요성)	추가 하드웨어(지문·카메라)나 별도 2차 인증 앱 조작 없이 **타이핑 행위 자체만으로 제로 마찰(Zero-Friction) 보조 생체 방어선**을 제공하기 위함

- **기존 서비스·연구 대비 차별점**: ① 고정된 단일 암호(`.tie5Roanl`) 전용 통계 판별기(Killourhy & Maxion, 2009; TypingDNA 고정형 API 등)와 달리 **임의 길이(`5 ≤ L ≤ 44+`)의 사용자 지정 비밀번호와 한글·영문 자유 타자연습 문장을 단일 신경망 구조로 통합 처리**하며, ② 오타 발생 시 재입력을 강제하던 기존 데모와 달리 **`Clean-Digraph Stitching` 으로 백스페이스 수정 문장을 그대로 수용**하고, ③ 원시 타건 타이밍을 외부 추론 서버로 보내지 않는 **`600 KB` 급 ONNX WebAssembly 클라이언트 추론 + 신호→바코드→2D PCA 판정 스냅샷 시각화**를 결합했습니다.

1.2 핵심 과업 정의 및 입·출력 명세

본 프로젝트는 공통 녹화·전처리 파이프라인과 동일 아키텍처의 `KeyPrintNet` 인코더 두 개(문장용 17채널, 비밀번호용 9채널)로 두 가지 행동 생체인식 과업을 해결합니다.

구분	[모드 1] 1:1 유출 암호 해커 침입 차단 (`Open-Set Verification`)	[모드 2] 1:N 타자연습 인물 자동 분류 (`Multi-User Identification`)
상황 가정	사칭자·매크로 봇이 **피해자의 정확한 비밀번호 문자열을 이미 알고 입력**한 상황	`N` 명의 사용자가 타자연습 문장을 각 5회씩 등록해 둔 뒤, 누군가 미경험 문장 2개를 타이핑한 상황
모델 입력 (Input)	`M` 개 연속 키 전이(`L = M − 1`)의 **9채널 타이밍 텐서** $X_9 \in \mathbb{R}^{(9\times L)}$ 및 **17채널 텐서** $X_{17} \in \mathbb{R}^{(17\times L)}$ + 키보드 대문자 입력 습관(`Shift` vs `CapsLock`)	영문·한글 타자연습 문장(두벌식 자모 키 이벤트)의 **17채널 시퀀스** $X_{17} \in \mathbb{R}^{(17\times L)}$ (타이밍 9 + 키 위치 7 + 누름 유효 1) + **행동 프로필 16종 통계**
인코더 출력	단위 초구면($\|z\|_2 = 1$) 상의 **64차원 타건 생체 지문 임베딩** $z_9, z_{17} \in \mathbb{R}^{64}$	길이 불변 **64차원 타건 생체 지문 임베딩** $z_{17} \in \mathbb{R}^{64}$

구분	[모드 1] 1:1 유출 암호 해커 침입 차단 (Open-Set Verification)	[모드 2] 1:N 타자연습 인물 자동 분류 (Multi-User Identification)
최종 출력 (Output)	길이 보정 듀얼 생체 일치도(0~100% , 기준 50%), **승인 (ACCESS GRANTED) / 차단(INTRUSION BLOCKED)**, 타수·홀드·롤오버·자동 갱신 요약 및 원리 탭 판정 스냅샷	등록된 N 명 전원에 대한 **실시간 일치도(0~100%) Top-1 ~ Top-N 순위표**, 행동 프로필 반영폭(±%p), **미등록 외부인(등록되지 않은 새로운 사용자) 기각** 및 경합 시 **판별 보류**

1.3 정량적 목표 설정 및 달성 현황 요약

학습과 임계값 선택에서 완전히 분리된 **미학습 피험자(Subject-Disjoint Unseen Test Split)** 기준 초기 목표와 누수·전처리 보정 후 최종 달성치는 다음과 같습니다.

- **[과업 1 · 1:1 해커 차단]** 미학습 피험자 10인 · 3,600 건 교차 세션 검증 쌍 기준 목표 **Test Macro F1 ≥ 0.85** , **ROC-AUC ≥ 0.92** , **EER ≤ 15.0%** → 제안 KeyPrintNet 17채널이 **ROC-AUC = 0.9318 (달성)**, **EER = 14.39% (달성)**, **Accuracy = 85.00%** 를 기록했고, **Test Macro F1 = 0.8491** (9채널 0.8446 , 통제 Vanilla1DCNN + 증강 + AS-Norm 0.8497) 로 F1 ≥ 0.85 목표에 **0.0009 근접 미달**했습니다(임계값을 검증셋에서만 고정 적용한 정직한 결과로 역조정하지 않음).
- **[과업 2 · 1:N 인물 분류]** 비중첩(stride = 25) 청크 기준 동일 문구 미학습 10인 분류 목표 **Top-1 ≥ 40%** , **Top-3 ≥ 88%** → **Top-1 = 48.26%** , **Top-3 = 88.60% (전 항목 달성)**; 영문 25자 문장 미학습 18인 분류 목표 **Top-1 ≥ 55%** , **Top-3 ≥ 88%** → **Top-1 = 62.41%** , **Top-3 = 91.01% (전 항목 달성)**.
- **[실시간 웹 서비스]** 추론 서버 없는 브라우저 로컬 WebAssembly **단일 추론 지연 ≤ 5.0 ms 내외**(실측 ~1.5~5.2 ms , Python 순방향 0.4~0.9 ms), ONNX 가중치 크기 **≤ 1.0 MB** (문장용 599 KB , 비밀번호용 592 KB , **달성**).

---

2. 데이터

고정 비밀번호 해커 차단(1:1)과 자연어 타자연습 문장 인물 식별(1:N)을 모두 실제 인간 타건 데이터로 학습·검증하기 위해, 학술적으로 검증된 **이중 공개 벤치마크 코퍼스(Dual Real-World Keystroke Corpora)**를 수집·구축하였습니다.

2.1 수집 데이터셋 명세 및 출처

데이터셋 구분	출처 및 라이선스	수집 프로토콜 및 텍스트 특성	원시 규모	정제 후 유효 규모	결측치 및 클래스 분포
① CMU Keystroke Dynamics Benchmark (DSL-StrongPasswordData.csv)	Killourhy & Maxion, *IEEE/IFIP DSN 2009* (CMU 공식 아카이브, 학술 공개 · freely available)	**51명**의 피험자가 최소 1일 이상 간격을 둔 **8개 독립 세션**에서 강건 암호 .tie5Roanl + Enter (11키 , L=10 전이)를 세션당 50회씩 반복 입력	20,400 행 × 34 열 (4.5 MB)	**20,400 시퀀스** (51명 × 400회)	결측치 **0건** , 전 피험자 정확히 **400건씩 보유한 완전 균형 분포**
② KeyRecs Multi-Session Transcription Dataset (KeyRecs-free-text.csv , KeyRecs-fixed-text.csv)	Zenodo DOI 10.5281/zenodo.7886742 (*CC-BY 4.0 오픈 라이선스*)	**99명 전원 2세션(198 세션)** 보유, 영문 문학 자유 필사(세션 1: Proust 소설 발췌, 세션 2: Murakami 소설 『해변의 카프카』 발췌) 및 고정 문자열 (vpwjkeurkb) 자연 타이핑	자유 필사 데이터 **562,596 행** (562,398 전이 + 198 세션 종단 홀드, 비이스케이프 " 타건 26 행 무손실 복구, 손상 0 행) + 고정문 19,772 행	비중첩 25자 (stride=25 , L=24) 무오타 청크 기준 통과 **96명** 의 문장 **11,105 건** + Train 고정문 13,588 건	세션 종단 단일 키 행(198 행)까지 홀드 복원 완료, 유효 96명 평균 115.7 개 비중첩 문장 보유

2.2 이중 코퍼스 선정 이유 및 탐색적 데이터 분석 (EDA)

- **왜 CMU + KeyRecs 이중 코퍼스를 결합했는가?**: CMU 데이터셋은 8개 세션에 걸친 **장기 숙련도 속도 드리프트**와 고정 암호 1:1 검증의 표준 비교 기준을 제공하지만 모든 샘플이 단일 암호(.tie5Roanl , L=10)에 고정되어 있어 임의 문장·키 위치 일

반화를 학습할 수 없습니다. 반대로 `KeyRecs`는 **99명의 대규모 자유 문학 필사(세션 간 서로 다른 소설 텍스트)**와 실제 오타(`Backspace`), 대문자 수정자(`Shift` / `CapsLock`), 멈춤 습관을 모두 담고 있어 `1:N` 타자연습 식별과 임의 비밀번호 전이 평가에 필수적입니다. 두 코퍼스를 결합함으로써 고정 암호의 정밀한 박자 판별력과 자유 문장의 키 배열·행동 일반화를 동시에 확보했습니다.

- **디그래프(`k_t → k_{t+1}`) 3대 물리 타이밍 변수 및 핵심 생체 특성**:
  - **키 홀드 시간 (`Hold`, `H_t = U_t − D_t`)**: 키 `k_t`를 누른 시점(`D_t`)부터 뗀 시점(`U_t`)까지의 지속 시간 (손가락 누름 탄성 습관 반영)
  - **키 간 눌림 간격 (`Down-Down`, `DD_t = D_{t+1} − D_t`)**: 현재 키를 누른 뒤 다음 키를 누르기까지의 리듬 간격 (전체 타자 템포 및 손가락 이동 거리 반영)
  - **키 간 비행 시간 (`Up-Down`, `UD_t = D_{t+1} − U_t = DD_t − H_t`)**: 앞 키에서 손을 뗀 후 다음 키를 누르기까지의 공중 체류 시간
  - **연타 롤오버(`Rollover`, `UD_t < 0`)**: 숙련된 타이피스트는 앞 키를 완전히 떼기 전에 반대 손이나 인접 손가락으로 다음 키를 먼저 누릅니다. CMU Train 분할 전이의 `10.99%`가 음수 비행시간(`UD_t < 0`)을 보이며, 사람마다 롤오버가 발생하는 키 쌍과 깊이가 일관된 반면 단순 매크로 봇이나 독수리 타법 사칭자는 `UD_t > 0`에 머무릅니다.
  - **세션 숙련도 속도 드리프트(`Temporal Speed Drift`)**: CMU 피험자의 세션 1 대비 세션 8 평균 입력 시간은 큰 폭으로 단축되며, 절대 타이밍(`ms`)만 비교하는 거리 모델이 본인을 침입자로 오차단(`FRR` 급증)하는 핵심 원인이 됩니다.

---

3. 데이터 전처리

원시 타건 로그의 극단치·파싱 왜곡을 제거하고 학습↔웹 서빙 간 수학적 괴리(Training-Serving Skew)를 원천 차단하기 위해 **4단계 도메인 특화 전처리 파이프라인**(`src/preprocessing.py`, `src/capture_sim.py`)을 구현하고, 웹 추론 엔진(`web/src/recorder.js`, `web/src/features.js`)에 **동일한 단일 수식과 Train 적합 통계량**을 이식했습니다.

3.1 적용한 4가지 핵심 전처리 기법

1. **원시 `H·DD` 보존 기반 생리학적 윈저라이징(Winsorizing) 및 무손실 이벤트 타임스탬프 복원**:
   - 실험 중 기침·머뭇거림으로 인해 CMU 전체 원시 데이터의 `DD.i.e`는 최대 `25.99초` (Train 분할 내 최대 `2.70초`, 왜도 `13.78`, 평균 첨도 `66.05`)의 우측 꼬리 분포를 보입니다. **오직 Train 분할 피험자**의 분위수와 생리학적 상·하한을 결합한 실제 코드 수식
     - `H ∈ [max(P0.05, 0.012s), min(P99.8, 0.450s)]` → 적합 경계 **`[0.0120s, 0.2203s]`**
     - `DD ∈ [max(P0.05, 0.015s), min(P99.5, 1.500s)]` → 적합 경계 **`[0.0150s, 1.2384s]`**
     - `UD ∈ [max(P0.05, −0.250s), min(P99.5, 1.400s)]` → 적합 경계 **`[−0.1283s, 1.1584s]`** 을 적용하되, **클리핑 전 원시 `H, DD`를 보존한 상태에서 `raw_UD = raw_DD − raw_H`를 먼저 계산한 뒤 `H, DD, UD`를 각각 독립 클리핑**하여 연타 롤오버 깊이가 왜곡되지 않도록 전 경로를 통일했습니다.
   - `KeyRecs-free-text.csv`는 줄 단위 인용 보호 파서로 비이스케이프 큰따옴표(`"`) 타건 `26`행과 세션 마지막 키 홀드 `198`행을 손실 없이 복원하고, 누적 눌림 시각 `t_down[i] = ∑_{j < i} DD_j`에서 `Shift` 음수 지연을 물리 시각으로 정렬합니다. 또한 `Backspace`·이상치 건너뜀 봉합 시 전이별 `(H_curr, H_next)` 및 관측 마스크 `(hvCurr, hvNext)`를 개별 보존하고 비연속 접합점(`stitchFlags[t] = true`)의 `Local_Accel`을 정확히 `0`으로 리셋합니다(모델 입력 `mean_H`는 대체·클리핑 후 고유 타건 평균, UI 표시 `holdMs`는 실측 홀드 평균으로 분리).
2. **디그래프 파생 피처 엔지니어링 (`4채널 → 9채널 타이밍 → 17채널`)**:
   - 당일 컨디션이나 숙련도에 따라 전체 타자 속도(WPM)가 달라져도 **각 키 구간의 상대적 박자 비율**은 유지된다는 원리에 착안해 시퀀스 내 평균 홀드(`H̄`) 및 평균 전환 간격(`D̄D`)으로 정규화한 5종의 생체 파생 채널과 8종의 키 배열·유효성 채널을 설계했습니다.

채널 인덱스	피처명	수식 정의	포착하는 행동 생체역학적 의미
`Ch 0`	`H_curr`	`H_t`	현재 키 `k_t` 의 클리핑(또는 터치 중앙값 대체) 후 누름 지속 시간
`Ch 1`	`H_next`	`H_{t+1}`	다음 키 `k_{t+1}` 의 누름 지속 시간
`Ch 2`	`DD`	`DD_t = D_{t+1} − D_t`	키 간 전환 눌림 간격 (절대 리듬 간격)
`Ch 3`	`UD`	`clip(raw_DD_t − raw_H_t)`	키 간 비행 시간 (양수: 공중 이동 / 음수: 키 중첩 롤오버)
`Ch 4`	**`Rel_H`** *(파생)*	`H_t / (H̄ + ε)`	**템포 불변 상대 홀드 비율** (타자 속도가 변해도 유지되는 손가락 탄성 패턴)
`Ch 5`	**`Rel_DD`** *(파생)*	`DD_t / (D̄D̄ + ε)`	**템포 불변 상대 전환 리듬** (단어 내 박자 분할 구조)
`Ch 6`	**`Duty_Ratio`** *(파생)*	`clip(H_t / max(DD_t, 0.02), 0, 5)`	전환 주기 대비 건반 점유율 (스타카토 vs 레가토 타법 구분)
`Ch 7`	**`Rollover_Mag`** *(파생)*	`max(0, −UD_t)`	**비선형 연타 롤오버 깊이** (양손 교차 연타 숙련도 핵심 지문)
`Ch 8`	**`Local_Accel`** *(파생)*	`DD_t − DD_{t−1}` (`t = 0` 또는 봉합점 `0`)	연속 철자 입력 시 **순간 가속·감속 궤적**
`Ch 9~12`	**`Row/Col_curr`** , **`Row/Col_next`** *(v2)*	`KeyboardEvent.code` 물리 행·열 좌표 / `(4, 13.5)`	같은 지연이라도 **어느 키 쌍인지**에 따른 손가락 접근 특성
`Ch 13`	**`Key_Dist`** *(v2)*	두 키 물리 좌표 사이 유클리드 거리	손가락 도약 거리
`Ch 14~15`	**`Same_Hand`** , **`Same_Finger`** *(v2)*	표준 운지 기준 같은 손(`1/0` , 미상 `0.5`)·같은 손가락 여부	양손 교차 연타 vs 단일 손가락 연속 도약 구분
`Ch 16`	**`Hold_Valid`** *(v2)*	`hvCurr_t ∧ hvNext_t` (`1` 또는 `0`)	두 키 모두 실측 홀드가 존재하는지 표시 (Z-Score 스케일링 제외)

- 한글은 두벌식 자모를 물리 스캔 코드(`ㅎ → KeyG` , `ㅏ → KeyK` …)로 변환해 동일한 좌표표를 공유하며, 9채널 비밀번호 인코더는 `Ch 0~8` 만 입력받습니다.

3. **피험자 완전 분리 분할(`Subject-Disjoint Split` , `seed=42`), 비중첩 창(`stride=25`) 및 분할별 역할 격리**:
   - 동일 인물의 타건이 학습과 평가에 함께 들어가는 데이터 누수를 막기 위해 **피험자 ID 단위로 완전 격리 분할**하고, 동일 세션 1:N 평가에서 등록 청크와 질의 청크 끝·첫 키가 겹치지 않도록 `25` **키 고정 비중첩 윈도우(`stride = 25` , `L = 24`)**만 추출했습니다.
     - **CMU (`51명` , `20,400` 건)**: `Train` **`35명 (68.6%, 14,000건)`** / `Validation` **`6명 (11.8%, 2,400건)`** / `Test` **`10명 (19.6%, 4,000건)`**
     - **KeyRecs 유효 청크 (`96명` , 자유문장 `11,105` 건 + Train 고정문 `13,588` 건)**: `Train` **`68명 (70.8%, 자유문장 7,971건 [71.8%] + 고정문 13,588건)`** / `Validation` **`10명 (10.4%, 1,055건 [9.5%])`** / `Test` **`18명 (18.8%, 2,079건 [18.7%])`**
     - **통합 규모**: `Train` **`103명 (35,559건)`** · `Validation` **`16명 (3,455건)`** · `Unseen Test` **`28명 (6,079건)`**
   - **역할 엄격 분리**: ① **`Train` 전용** — 클리핑 경계, 채널별 Z-Score 스케일러(`Ch 0~15`), 터치 홀드 대체 중앙값(`88.0ms`), 행동 특성 2-공분산 분포(`σ², τ², δ²`), AS-Norm 배경 코호트; ② **`Validation` 전용** — 체크포인트·인코더 선택, 행동 융합 가중치(`β`), 전 판정 임계값(`τ`) 및 듀얼 보정 통계; ③ **`Test` 전용** — 확정된 파이프라인의 최종 성능 측정.

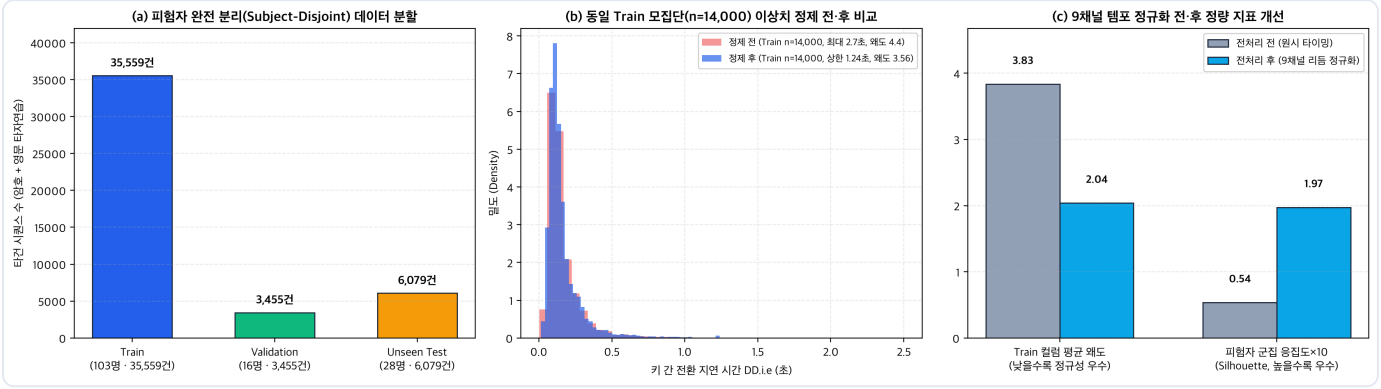
4. **원시 타이밍 탄성 템포 워핑(Smooth Tempo Warping) 및 터치 모사 증강**:

- 학습 미니배치마다 **클리핑 전 원시** `raw_H, raw_DD` 에 전체 속도 배율 `α ~ U(0.80, 1.25)` 와 사인파 가속 곡선 `w(t) = 1 + 0.06·sin(πt/L + φ)` , 미세 지터를 먼저 적용한 뒤 `raw_UD = aug_DD − aug_H` 산출 → 독립 클리핑 → 파생 피처 재계산 순서를 수행합니다. 또한 미니배치의 `18%` 는 홀드를 Train 중앙값(`88ms`)으로 대체하고 `Hold_Valid = 0` 을 부여해 물리 키보드와 모바일 소프트 키보드를 단일 모델로 처리합니다.

3.2 전처리 전·후 동일 모집단 정량 비교 및 단계별 실루엣 분해

모집단 혼재를 배제한 **동일 모집단(`CMU Train 35명, n = 14,000`)** 기준 왜도·첨도 감소 효과와 **미학습 테스트 피험자(`CMU Test 10명, n = 4,000`)**의 전처리 단계별 코사인 군집 응집도(`Silhouette`) 변화는 다음과 같습니다.

정량 비교 항목 (비교 모집단 명시)	전처리 전 (원시 타이밍)	전처리 후 (전처리 완료)	단계별 분해 및 개선 효과 요약
타이밍 컬럼 평균 왜도 (`Mean Skewness` ↓, `Train n=14,000`)	`3.8344`	**`2.0389`**	동일 Train 모집단 기준 비대칭 꼬리 왜도 **46.8% 감소**
최대 단일 컬럼 왜도 (`DD.i.e Max Skewness` ↓, `Train n=14,000`)	`13.7780` (Train 최대 `2.70s` / 전체 `25.99s`)	**`4.1107`** (상한 `1.24s`)	기침·일시정지 극단치 제거로 학습 그래디언트 폭주 방지
타이밍 컬럼 평균 첨도 (`Mean Kurtosis` ↓, `Train n=14,000`)	`66.0474`	**`8.1443`**	동일 Train 모집단 기준 극단적 첨도 **87.7% 감소**
미학습 피험자 코사인 실루엣 (`Silhouette` ↑, `Test 10명`)	`0.0537` (원시 4ch)	**`0.1974`** (표준화 9ch)	원시 4ch(`0.0537`) → 클리핑 4ch(`0.0658`) → 표준화 4ch(`0.1627`) → **파생 9ch(`0.1974` , `3.68배` 향상)** *(고정 암호 상수 키 좌표를 포함한 17ch는 `0.1686`)*



---

4. 모델

동일한 피험자 분할과 `35 Epochs` 학습 예산 위에서 전통 통계 모델, 일반 분류용 딥러닝 베이스라인(순수 기본형 및 동일 증강·`AS-Norm` 통제 변형), 그리고 본 프로젝트가 제안하는 생체 메트릭 러닝 아키텍처(`KeyPrintNet` 9채널·17채널)를 **모두 처음부터 직접 구현·학습**하여 객관적으로 비교했습니다(`src/models.py` , `src/train_and_evaluate.py`).

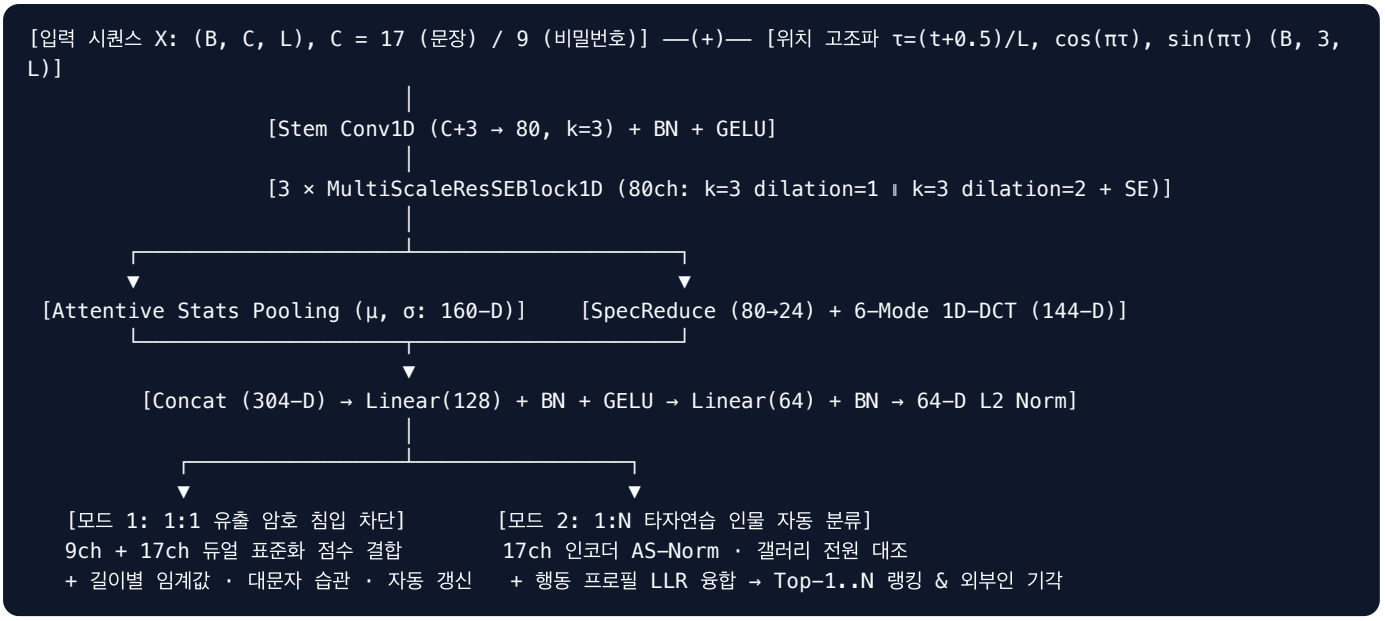
4.1 비교 모델 선정 이유 및 아키텍처 명세

- **베이스라인 선정 근거**: ① **`ScaledManhattanDetector`** 는 CMU 벤치마크 원 논문(Killourhy & Maxion, 2009)에서 14종 이상치 탐지기 중 최상위 성능을 기록한 키스트로크 분야의 대표 고전 통계 기준선이며, ② **`Vanilla1DCNN`** 은 위치 인코딩·스펙트럼 풀링·각도 마진 손실 없이 순수 1D 합성곱과 전역 평균 풀링(`GAP`)만으로 학습했을 때 도달할 수 있는 표준 딥러닝 기준선을 측정합니다. 또한 증강과 후처리 효과를 아키텍처 효과와 분리하기 위해 **`Vanilla1DCNN + 동일 증강`** 및 **`Vanilla1DCNN + 동일 증강 + AS-Norm`** 통제 변형을 함께 학습·평가했습니다.

구분	모델명	핵심 구조 (`src/models.py` 실제 구현 기준)	손실 함수 · 증강 · 보정 방식	가변 길이(`L`) 지원	파라미터 / 크기
Baseline 1	**`ScaledManhattanDetector`**	등록 샘플 평균 `μ_u` 및 경험적 베이즈 수축(`λ = 0.35`) 절대편차 `s_u` 기반 정규화 맨해튼 거리	Train 코호트 사전편차 적합 + 5-shot 등록 템플릿	불가 (고정 길이 벡터 전용)	통계 파라미터
Baseline 2	**`Vanilla1DCNN`** *(기본 / 통제 변형)*	위치 정보 없는 3층 `Conv1D(17 → 32 → 64 → 64, k=3)` + `BN` + `ReLU` + `Global Average Pooling` → `Linear(64 → 64)` L2 정규화	표준 `Softmax Cross-Entropy` (`35 Epochs` , `AdamW` ; 기본형은 무증강·무보정, 통제형은 동일 증강· `AS-Norm` 적용)	가능 (평균 풀링 시 위치 소실)	`~26K` (`116 KB`)
Proposed (최종 채택)	**`KeyPrintNet`** **(17ch 문장용 / 9ch 암호용)**	**연속 위치 고조파(`τ = (t+0.5)/L`) + 다중 스케일 팽창 잔차 SE-Conv1D(`k=3, dilation=1,2` , 3블록, 80ch) + 6모드 1D-DCT(`80→24ch` , `144-D`) & ASP(`160-D`) 결합(`304-D → 128-D → 64-D`) + AS-Norm / 듀얼 결합**	**도메인별 클래스 분리(`171` 클래스) `Sub-Center CosFace(K=2, s=16, m=0→0.08)` + `Supervised Contrastive Loss`** (`35 Epochs` , 템포·터치 증강)	**완벽 지원** (신규 암호 `12자+` ·샘플 `10자` ~ 문장 `45자+` , ONNX 동적 축)	문장용 17ch **`599 KB`** / 암호용 9ch **`592 KB`**

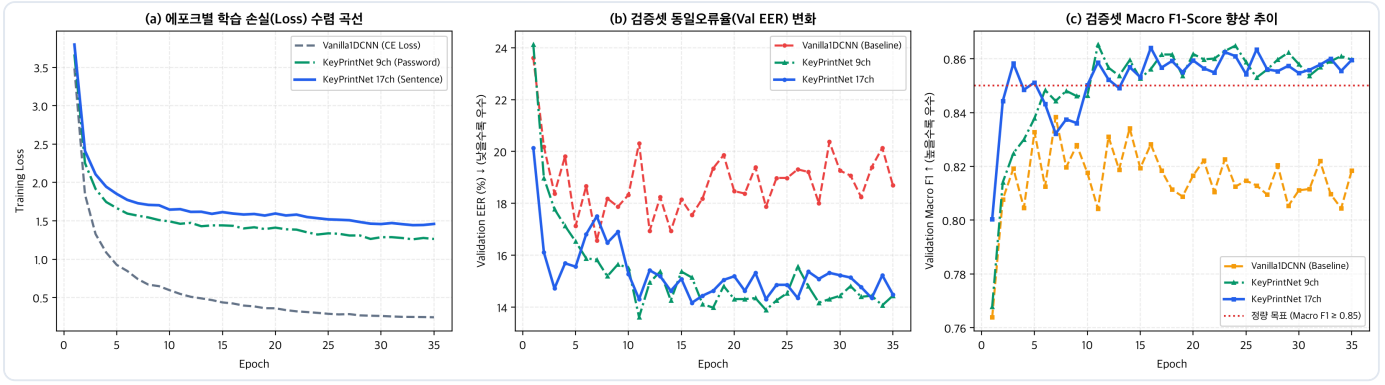
4.2 제안 모델 `KeyPrintNet` 의 4대 핵심 설계 근거

1. **동적 길이 호환 연속 상대 위치 고조파 주입 (`Continuous Positional Harmonics`)**:
   - 일반 `Global Average Pooling` (`Vanilla1DCNN`)은 문장 앞부분과 뒷부분의 타이밍 순서를 뒤섞어도 동일한 평균값을 내어 **어느 구간에서 지연·롤오버가 발생했는지 소실**됩니다. `KeyPrintNet` 은 임의 시퀀스 길이 `L` 에 대해 셀 중심 정규화 좌표 `τ_t = (t + 0.5) / L ∈ (0, 1)` (`t = 0, …, L-1`) 및 삼각 고조파 `[τ_t, cos(πτ_t), sin(πτ_t)]` 3채널을 입력(`17ch` 또는 `9ch`)에 결합하여, ONNX 동적 축 상수 폴딩 없이 초두·중반·종결부의 박자 위치를 보존합니다.
2. **병렬 팽창 잔차 Squeeze-and-Excitation 블록 (`MultiScaleResSEBlock1D`)**:
   - 각 블록(은닉 `80ch` , 총 3블록) 내부에서 동일 커널 크기 `k = 3` 의 **국소 분기(`dilation = 1` , `40ch` , 인접 디그래프)**와 **광역 팽창 분기(`dilation = 2` , `40ch` , 수용장 `5` 의 트라이그래프·음절 리듬)**를 병렬 결합한 뒤 `1×1` 점별 합성곱과 SE 채널 어텐션(`80 → 20 → 80`)을 적용해 식별력이 높은 파생·위치 채널에 가중치를 부여합니다.
3. **스펙트럼 시간 피라미드(`1D-DCT`) + 주의집중 통계 풀링 (`ASP + Spectral Pooling`)**:
   - 채널별 어텐션 가중 평균·표준편차(`80 × 2 = 160-D`)에 더해, `1×1 Conv(80 → 24)` 축소 특징에 순수 행렬곱 (`torch.bmm`) 기반 **6모드 이산 코사인 변환(`1D-DCT` , `24 × 6 = 144-D`)** 계수를 결합(`총 304-D → Linear 128-D → 64-D L2 정규화`)하여 시퀀스 길이가 달라져도(`L = 5~44+`) 문장 전체의 저주파·중주파 시간 엔벨로프를 포착합니다.
4. **도메인 간 클래스 인덱스 분리 `Sub-Center CosFace` + `AS-Norm` 코호트 교정**:
   - 고정 암호(`CMU 35명` , `KeyRecs-fixed 68명`)와 자유 필사 문장(`KeyRecs-free 68명`)은 위치별 철자 고정 여부가 다르므로, 백본 가중치는 100% 공유하되 단일 `SubCenterCosFaceHead` 내부의 **클래스 프로토타입 인덱스를 세 코퍼스 간 비중첩 분리(`35 + 68 + 68 = 171` 클래스, 서브센터 `K = 2`)**하고 코사인 마진(`s = 16.0` , `m = 0 → 0.08` 20에포크 코사인 램프업)과 `SupCon` 손실을 결합했습니다. 추론 시에는 Train 코호트 중심점 대비 상대 Z-점수를 산출하는 **AS-Norm** 과 듀얼 표준화 결합으로 유사 속도 사칭자(`Wolf`) 오허용을 억제합니다.



4.3 학습 수렴 곡선 (`Vanilla1DCNN` vs `KeyPrintNet` , `35 Epochs` 동일 예산)

공식 학습 실행(`run_id: kp-v4-20261113T130002Z`)의 에포크별 로그 기준, 제안 모델 `KeyPrintNet` (17채널)은 기본 `Vanilla1DCNN` 대비 **더 낮은 검증 동일오류율(`Val EER` 최소 `14.17%` vs `16.57%` ; 9채널 `KeyPrintNet` 은 최소 `13.61%`)과 더 높은 검증 `Macro F1` (`0.8641` vs `0.8384` , `Val AUC 0.9377` vs `0.9086`)**으로 안정 수렴합니다.



---

5. 평가 결과

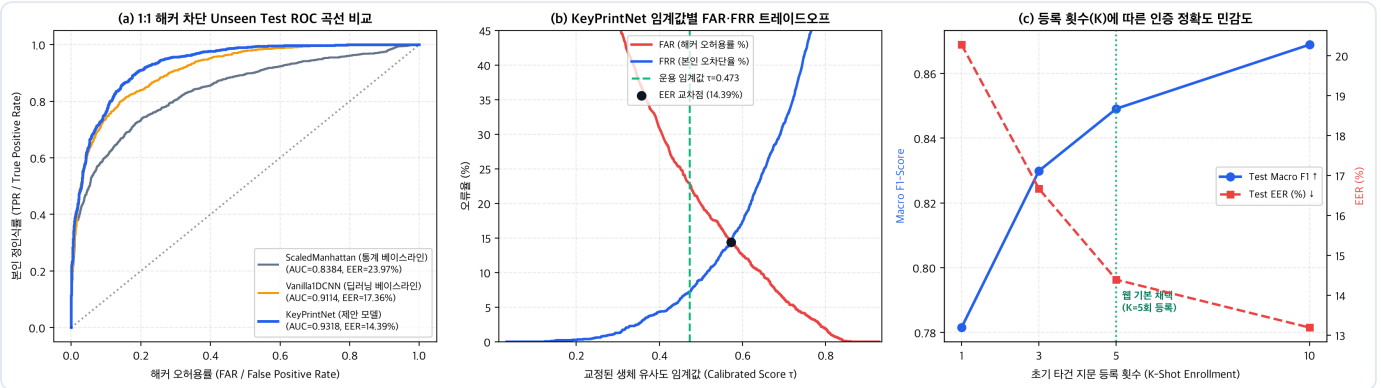
- **평가 프로토콜 및 테스트셋 재평가 고지**: 모든 평가는 학습·전처리 적합(`Train 103명`) 및 모델·가중치·임계값 선택(`Validation 16명`)에서 완전히 격리된 **Unseen Test 피험자(`CMU 10명` + `KeyRecs 18명` = 총 `28명`)**만을 대상으로 수행되었습니다. 모든 결정 임계값(`τ`), 융합 가중치(`β`), 듀얼 보정 파라미터는 오직 `Validation` 분할에서만 확정했습니다. 다만 이전 개발 반복 과정에서 동일 벤치마크 테스트셋 지표를 확인했던 이력이 존재하므로, 아래 수치는 **전처리(`UD` 원시 보존·봉합 홀드 정합화) 및 선택 누수(`val` -only 고정)를 모두 수정한 뒤 처음부터 재학습한 모델의 기존 테스트셋 재평가 결과**임을 투명하게 밝힙니다.

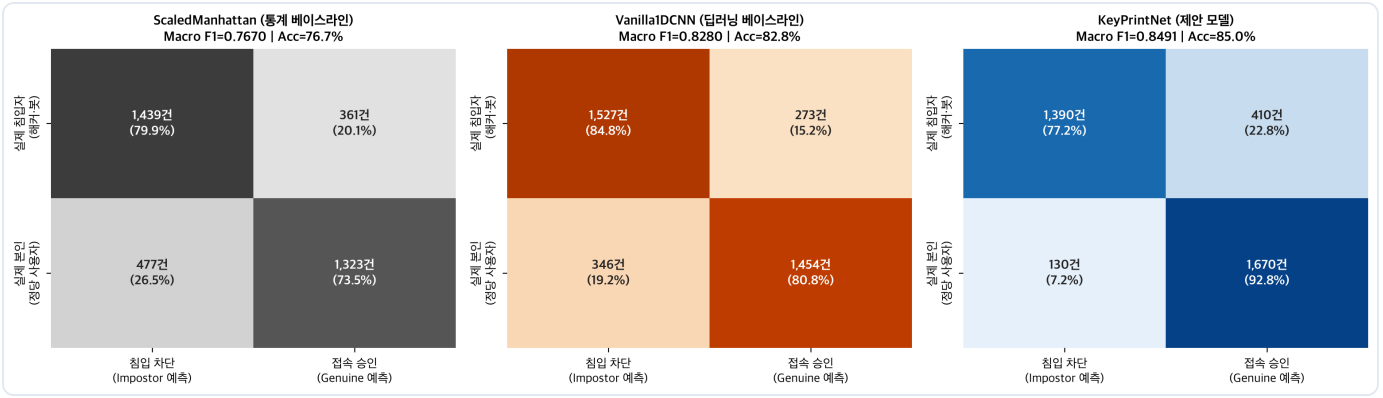
5.1 [과업 1] 1:1 유출 암호 해커 침입 차단 성능 비교 (Unseen 10명 · 3,600 검증 쌍)

각 미학습 피험자의 5회 등록 템플릿에 대해 **타 세션 본인 타건(`1,800` 건)** vs **동일 비밀번호(`.tie5Roanl`)를 정확히 입력한 공격 표본 `1,800` 건(`실측 타인 인간 타건 1,600건` + `합성 매크로 봇 200건`)**을 검증한 결과입니다.

모델 (Model , 전 모델 35 Epochs 동일 예산)	Test Macro F1 ↑	ROC-AUC ↑	PR-AUC ↑	EER ↓	Accuracy ↑	전체 FAR ↓	실측 타인 FAR (1,600 건) ↓	매크로 봇 FAR (200 건) ↓	본인 FRR (1,800 건) ↓	FAR@FRR=5% ↓
ScaledManhattan (통계 베이스라인)	0.7670	0.8384	0.8584	23.97%	76.72%	20.06%	20.31%	18.00%	26.50%	72.28%
Vanilla1DCNN (기본: 무증강·무보정)	0.8280	0.9114	0.9114	17.36%	82.81%	**15.17%**	**16.44%**	5.00%	19.22%	39.56%
Vanilla1DCNN + 동일 증강 (통제 1)	0.8333	0.9129	0.9165	16.03%	83.33%	17.72%	19.44%	**4.00%**	15.61%	41.44%
Vanilla1DCNN + 동일 증강 + AS-Norm (통제 2)	**0.8497**	0.9247	**0.9300**	14.56%	84.97%	17.11%	18.12%	9.00%	12.94%	39.94%
KeyPrintNet 9채널 (순수 타이밍 인코더)	0.8446	0.9216	0.9216	15.03%	84.47%	18.00%	19.31%	7.50%	13.06%	37.22%
KeyPrintNet 17채널 (제안 아키텍처)	0.8491	**0.9318**	0.9281	**14.39%**	**85.00%**	22.78%	25.06%	4.50%	**7.22%**	**27.11%**

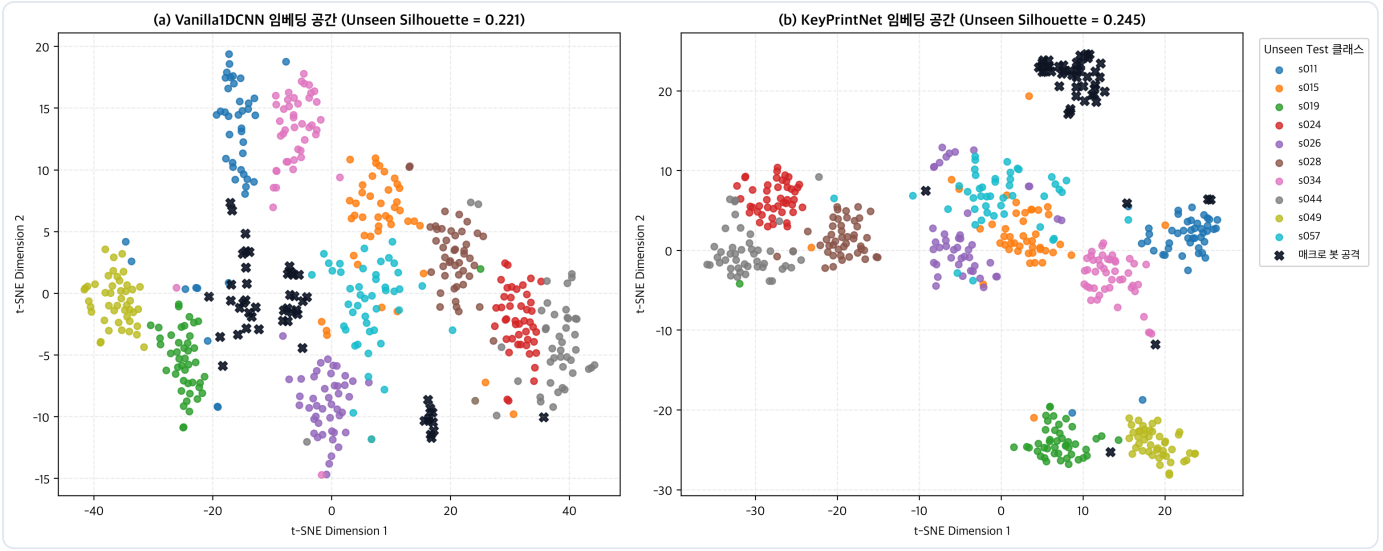
- **단계별 통제 비교 해석**: Vanilla1DCNN 에 동일 템포·터치 증강(EER 17.36% → 16.03%)과 AS-Norm (→ 14.56%)을 차례로 더하면 단순 3층 합성곱도 고정 암호에서 강한 기준선(Macro F1 = 0.8497)에 도달합니다. 그 위에서 제안 아키텍처 KeyPrintNet 17채널은 임계값 무관 순위 지표인 **ROC-AUC (0.9247 → 0.9318)**, **EER (14.56% → 14.39%)**, **Accuracy (85.00%)**, 그리고 실용 보안 핵심 지표인 **FRR = 5% 지점의 침입자 통과율(FAR@FRR=5%: 39.94% → 27.11% , −12.83%p)**에서 전 모델 중 최우수 성능을 달성했습니다. 다만 검증셋(6명) 최적 임계값(τ = 0.4727)을 테스트셋에 그대로 고정 적용했을 때 동작점이 낮은 본인 거부율(FRR 7.22%) 쪽에 놓이면서 Macro F1 은 0.8491 (목표 0.85 대비 −0.0009)을 기록했습니다.
- **실측 인간 침입자(1,600 건) vs 합성 매크로 봇(200 건) 분리 분석**: 제안 KeyPrintNet 17채널은 일정 간격·무롤오버 특성을 갖는 매크로 봇에 대해 **ROC-AUC = 0.9881 , EER = 6.14% (FAR = 4.50%)**로 거의 완벽히 차단하며, 난이도가 훨씬 높은 실측 인간 타건(1,600 건) 단독 비교에서도 **ROC-AUC = 0.9248 , EER = 15.71%** (9채널 EER = 15.77%)를 유지합니다.
- **등록 횟수(K-Shot) 민감도(17채널)**: 1회 등록 F1 = 0.7815 (EER = 20.28%) → 3회 F1 = 0.8299 (EER = 16.67%) → 5회 F1 = 0.8491 (EER = 14.39%) → 10회 **F1 = 0.8689 (ROC-AUC = 0.9425 , EER = 13.19%)**로 등록 샘플이 누적될수록 단조 향상됩니다.





5.2 미학습 피험자 임베딩 공간(t-SNE) 군집 품질 비교

미학습 테스트 피험자 10인과 자동화 매크로 봇 공격 샘플을 64차원 임베딩 공간에서 t-SNE로 투영한 결과, `KeyPrintNet`은 인물별 클러스터 응집도(`Unseen Silhouette = 0.245` vs `Vanilla1DCNN = 0.221`)를 높이며 매크로 봇 공격(`X` 마커)을 인간 피험자 군집 외곽으로 분리해 냅니다.



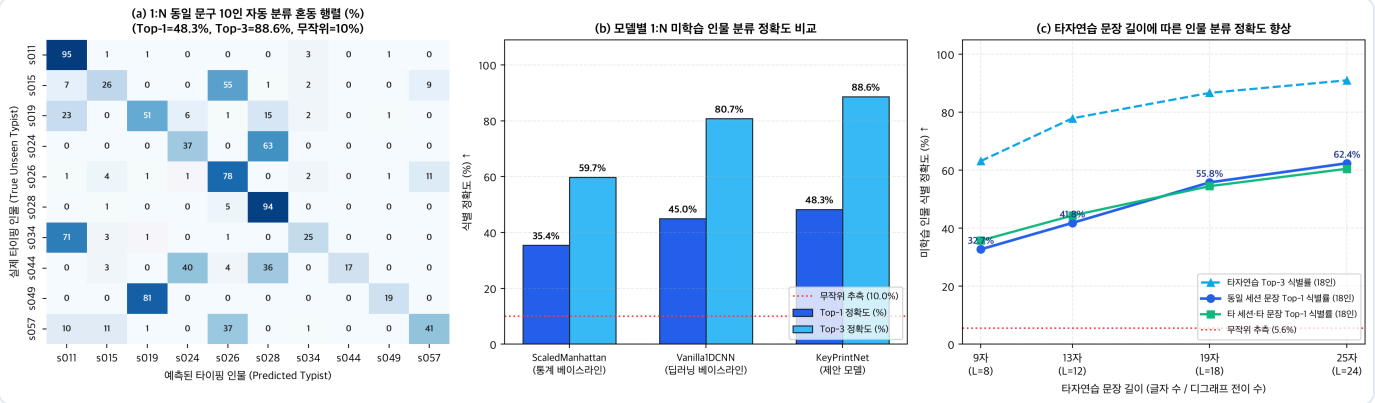
5.3 [과업 2] 1:N 타자연습 인물 자동 분류("Who Is Typing?") 성능 평가

미학습 사용자가 비중첩(`stride = 25`) 연습 문장 5개를 등록한 뒤, 블라인드 타건 1회만으로 작성자를 맞추는 다중 인물 분류(`1:N Identification`) 결과입니다.

평가 시나리오 및 모델 (`stride=25` 비중첩 청크)	미학습 후보 인물 수 (무작위 확률)	검증 쿼리 수	Top-1 정확도 ↑	Top-3 정확도 ↑	1:N Macro F1 ↑
동일 문구 10인 분류 — `ScaledManhattan`	10명 (10.0%)	3,500건 (타 세션)	35.40%	59.74%	0.2851
동일 문구 10인 분류 — `Vanilla1DCNN` (무증강)	10명 (10.0%)	3,500건 (타 세션)	44.97%	80.74%	0.4095
동일 문구 10인 분류 — `Vanilla1DCNN + 증강 + AS-Norm`	10명 (10.0%)	3,500건 (타 세션)	**48.69%**	88.17%	**0.4631**
동일 문구 10인 분류 — `KeyPrintNet` 9채널	10명 (10.0%)	3,500건 (타 세션)	41.91%	83.31%	0.3837
동일 문구 10인 분류 — **`KeyPrintNet` 17채널 (제안)**	10명 (10.0%)	3,500건 (타 세션)	48.26%	**88.60%**	0.4443
영문 문장(25자) 18인 분류 — `ScaledManhattan`	18명 (5.56%)	923건 (동일 세션)	25.68%	57.53%	0.2393

평가 시나리오 및 모델 (`stride=25` 비중첩 청크)	미학습 후보 인물 수 (무작위 확률)	검증 쿼리 수	Top-1 정확도 ↑	Top-3 정확도 ↑	1:N Macro F1 ↑
영문 문장(25자) 18인 분류 — `Vanilla1DCNN` (무증강)	18명 (5.56%)	923건 (동일 세션)	59.48%	89.49%	0.5787
영문 문장(25자) 18인 분류 — `Vanilla1DCNN + 증강 + AS-Norm`	18명 (5.56%)	923건 (동일 세션)	57.42%	89.82%	0.5529
영문 문장(25자) 18인 분류 — `KeyPrintNet` 9채널 (동일 세션)	18명 (5.56%)	923건 (동일 세션)	58.94%	**91.44%**	0.5597
영문 문장(25자) 18인 분류 — **`KeyPrintNet` 17채널 (동일 세션)**	18명 (5.56%)	923건 (동일 세션)	**62.41%**	91.01%	**0.6058**
영문 문장(25자) 18인 분류 — `KeyPrintNet` 9채널 (타 세션·타 소설)	18명 (5.56%)	1,066건 (Proust→Murakami)	53.85%	86.59%	0.4924
영문 문장(25자) 18인 분류 — **`KeyPrintNet` 17채널 (타 세션·타 소설)**	18명 (5.56%)	1,066건 (Proust→Murakami)	**60.51%**	**89.96%**	**0.5633**

- **동일 문구·자유 문장 전 구간 목표 달성**: 완전히 같은 문자열을 친 10인 타 세션 쿼리 `3,500` 건에서 17채널 `KeyPrintNet` 은 **`Top-1 = 48.26%`**, **`Top-3 = 88.60%`**로 목표(`40%` / `88%`)를 모두 충족했습니다. 자유 문장에서는 위치 정보를 무시하는 `Vanilla1DCNN` 에 템포 증강· `AS-Norm` 을 더하면 오히려 동일 세션 정확도가 `59.48% → 57.42%` 로 소폭 하락하는 반면, 위치 고조파와 키 배열 좌표를 갖춘 **`KeyPrintNet` 17채널은 동일 세션 `Top-1 = 62.41%` (`Macro F1 = 0.6058`)로 전 모델 1위**를 기록했습니다.
- **문장 길이에 따른 식별력 향상 & 타 문학 작품 전이**: 글자 수가 `9자(L=8)` → `13자(L=12)` → `19자(L=18)` → `25자(L=24)` 로 길어질수록 18인 분류 Top-1이 동일 세션 **`32.72% → 41.82% → 55.80% → 62.41%`** (교차 세션 **`35.74% → 44.37% → 54.50% → 60.51%`**, 무작위 대비 **10.9~11.2배**)로 상승합니다. 특히 세션 1(Proust)로 등록하고 **다른 날 전혀 다른 Murakami 소설을 친 교차 세션에서 17채널이 9채널 대비 `+6.66%p` (`53.85% → 60.51%` , Top-3 `86.59% → 89.96%`)** 높은 정확도를 달성해 키 위치 채널의 효과를 입증했습니다.

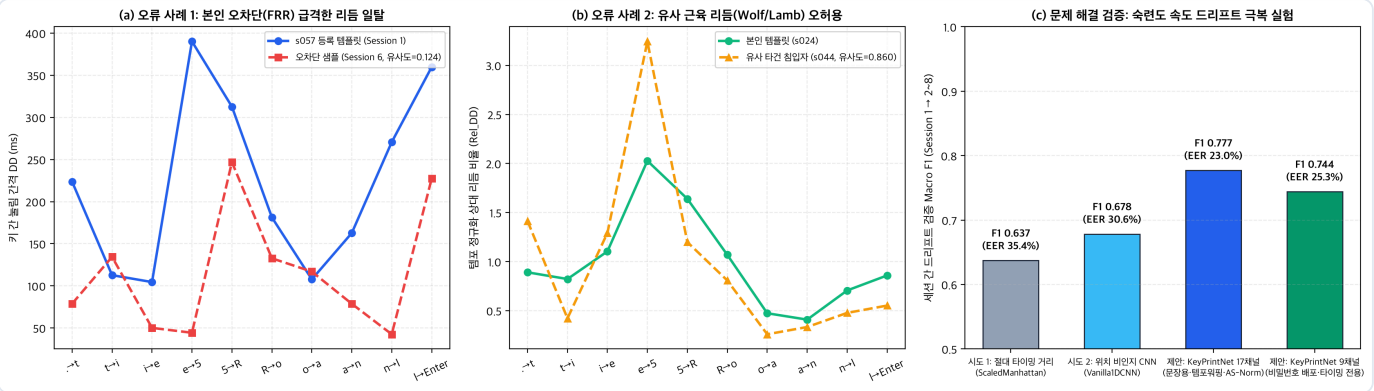


5.4 실제 오류 사례(Error Analysis) 및 세션 속도 드리프트 극복 검증

미학습 테스트셋(인간 검증 쌍 기준 `FR = 130` 건, `FA = 401` 건)에서 발생한 최악 오탐 사례를 파형 단위로 역추적하여 원인을 규명했습니다.

1. **오류 사례 1 — 본인 오차단(`FRR` 극단 사례, 패널 a)**: 피험자 `s057` 은 Session 1 초기 등록 대비 숙련된 Session 6(인덱스 `3876`)에서 대문자·숫자 전환 구간의 머뭇거림이 사라지고 박자 비율 구조 자체가 재편되면서 일치도가 `0.1241` 까지 하락해 거부되었습니다. 이는 §5.6의 **고신뢰 등록 리듬 자동 갱신** 도입의 직접적인 동기가 되었습니다.
2. **오류 사례 2 — 유사 근육 리듬 침입자(`Wolf/Lamb FAR` 사례, 패널 b)**: 침입자 `s044` 가 정당 소유자 `s024` (Session 5 등록, 인덱스 `2938`)와 주요 전환 구간에서 우연히 유사한 상대 박자 피크(`Rel_DD`)와 홀드 길이를 형성해 점수가 `0.8604` 까지 상승했습니다. 웹 UI의 `엄격` 민감도 및 `12자 이상` 길이별 임계값(§5.7)으로 이러한 경계선 침입을 줄일 수 있습니다.

3. **숙련도 속도 드리프트 극복 실험(패널 c): "Session 1(초기 미숙련) 5회만 등록 → Session 2~8(장기 숙련 고속화) 검증"** 조건에서 `ScaledManhattan` (`F1 = 0.6374` , `EER = 35.44%` , `FRR = 42.44%`)과 기본 `Vanilla1DCNN` (`F1 = 0.6784` , `EER = 30.56%` , `FRR = 41.11%`) 대비 **`KeyPrintNet` 17채널은 `F1 = 0.7773` (`EER = 23.00%` , `ROC-AUC = 0.8619` , `FRR = 13.78%`)**, 9채널은 `F1 = 0.7442` (`EER = 25.33%` , `FRR = 22.22%`)를 기록하여 통계 베이스라인 대비 본인 오차단율(`FRR`)을 **`28.66%p`** 단축했습니다.



5.5 v2 확장: 키 위치 채널 · 행동 프로필 · 모바일(터치) 입력

평가 방식 (웹과 동일하게 재현): 미학습 KeyRecs 시험 인원 18명의 원시 키 이벤트를 브라우저 녹화기의 Python 복제본(`src/capture_sim.py`)에 그대로 흘려 넣어, 웹과 똑같이 오타·백스페이스 봉합이 들어간 `68자` 캡처를 만들고 **3개로 등록 → 1개로 판별**했습니다(AS-Norm `top-16`). 쿼리는 같은 날 다른 문장(`559건`)과 다른 날 다른 글(`647건`) 두 종류이며, 터치는 동일 재생에서 누름 시간을 제거한 **시뮬레이션**입니다(실제 스마트폰 데이터 아님).

입력	인코더 / 점수	같은 날 다른 문장 Top-1 ↑	다른 날 다른 글 Top-1 ↑	다른 날 Top-3 ↑
키보드	9채널 (타이밍 전용)	`72.81%`	`61.36%`	`90.26%`
키보드	17채널 (키 위치 추가)	`84.44%`	`77.28%`	`94.59%`
키보드	**17채널 + 행동 프로필 (`β = 0.15` , 웹 배포)**	**`85.87%`**	**`81.30%`**	**`96.45%`**
키보드	행동 프로필 단독	`33.99%`	`32.61%`	`64.45%`
터치 (시뮬)	9채널 (누름 시간 중앙값 대체)	`47.41%`	`42.04%`	`73.42%`
터치 (시뮬)	17채널 (터치 모사 학습)	**`59.57%`**	**`55.49%`**	**`83.93%`**
터치 (시뮬)	**17채널 + 행동 프로필 (`β = 0.30` , 웹 배포)**	`59.21%`	`54.87%`	`81.30%`

- **키 위치 채널**: 다른 날·다른 글 Top-1이 **`+15.92%p`** (`61.36% → 77.28%` , 같은 날 `72.81% → 84.44%`) 상승했습니다. 같은 전환 지연이라도 손가락 이동 거리·같은 손 여부에 따라 생체 의미가 달라짐을 모델이 포착한 결과입니다.
- **행동 프로필**(백스페이스 비율·연속 삭제 길이·오타 인지/재개 지연·백스페이스 홀드, Shift/CapsLock 전략·Shift 선행 시간, 멈춤 비율, 스페이스 전후 리듬 등 16개 점수 특성): 단독으로는 `32.61%` 지만 17채널 임베딩과 결합하면 키보드 다른 날 Top-1이 **`77.28% → 81.30%`** (같은 날 **`85.87%`** , 다른 날 Top-3 **`96.45%`**)로 향상됩니다. 각 특성은 Train 68명에서 추정한 개인 내 분산(σ^2)·개인 간 분산(τ^2)·세션 드리프트(δ^2) 기반 **two-covariance 로그우도비(LLR)**로 점수화되므로 관찰 횟수가 적은 특성은 영향이 자동으로 0에 수렴합니다(개인 간 식별비 τ^2/σ^2 상위: CapsLock 비율 `4.6` , 백스페이스 홀드 `0.26` , Shift 홀드 `0.23` , 오타 인지 지연 `0.18`).
- **융합 가중치(`β`)는 오직 Validation 분할에서만 선택해 고정 배포했습니다** (`점수 = sigmoid(logit(s) + β·LLR)`): 비밀번호 `β = 0.0` (검증 AUC `0.9377` → `β=0.2` 적용 시 `0.9331` 로 하락), 키보드 문장 `β = 0.15` , 터치 문장 `β = 0.30` (검증 다른 날 Top-1 `65.00% → 74.44%` 로 상승해 채택). 터치에서는 검증셋의 큰 상승이 테스트셋(`55.49% → 54.87%` , `-0.62%p`)에서 재현되지 않았으나, 테스트 결과를 보고 `β` 를 되돌리지 않는 무누수 원칙에 따라 `β = 0.30` 을 그대로 웹에 배포했습니다.

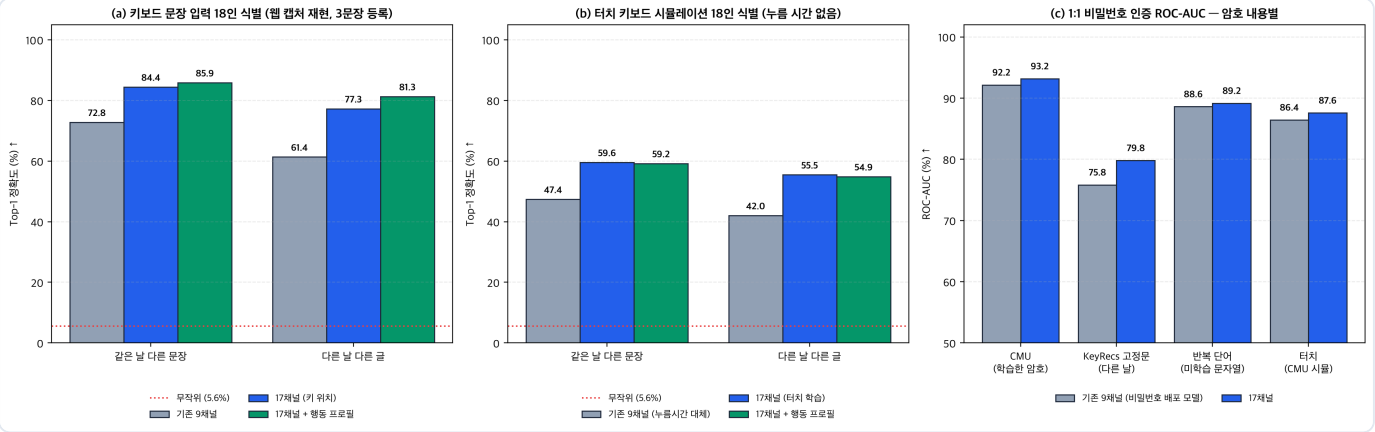
단일 인코더 비밀번호 전이 비교 — 오직 `Validation` 만으로 선택 후 순수 `Test` 에서 검증: 웹 비밀번호는 사용자가 직접 정한 미학습 문자열이므로, 먼저 **`Validation` 피험자(`10명`)에서의 전이 손실 비용(`9채널 0.6203 < 17채널 0.7063` ; 검증 반복 단어에서 9채널 `AUC 0.9399 · FAR 37.12%` vs 17채널 `AUC 0.9164 · FAR 49.24%`)만으로 단일 비밀번호 기준 인코더를 9채널로 확정**한 뒤, `Validation` 과 병합하지 않은 순수 `Test` 분할에서 평가했습니다.

순수 `Test` 평가 조건 (`Validation` 미병합)	9채널 AUC	17채널 AUC	배포 임계값에서 침입자 통과율 (FAR : 9채널 / 17채널)
CMU `.tie5Roanl` (학습한 암호, 미학습 `10명`)	`0.9216`	**`0.9318`**	**`18.00%`** / `22.78%`
CMU 터치 시뮬레이션 (미학습 `10명`)	`0.8644`	**`0.8758`**	**`17.72%`** / `26.78%`
KeyRecs 고정문 `vpwjkeurkb` 다른 날 (학습한 문자열, 미학습 `18명`)	`0.7583`	**`0.7982`** (`+0.040` , 95% CI `[+0.030, +0.050]`)	**`30.75%`** / `36.86%`
반복 단어 = 학습에 없던 문자열 (`because` · `things` · `people` , 미학습 `18명` , 본인 `84` / 침입 `396` 건)	`0.8862`	**`0.8917`** (`+0.006` , 95% CI `[−0.024, +0.034]`)	**`33.59%`** / `42.68%`

- 순수 `Test` 에서도 17채널 단독 AS-Norm은 학습에 없던 문자열에서 점수 분포가 위로 밀려 **배포 임계값 기준 침입자 통과율 (`FAR`)이 `33.59%` (9채널) 대 `42.68%` (17채널)로 `+9.09%p` 높게** 나타났습니다. 이에 따라 단일 AS-Norm 기준선에서는 9채널이 더 안정적이었으며, 나아가 §5.6에서는 코호트 없이 **9채널과 17채널의 원시 코사인 유사도를 각각 사칭 분포로 표준화해 합산하는 듀얼 인코더 점수**로 두 인코더의 장점을 결합했습니다(`src/eval_password_calibration.py`).

모바일 등록 실패("추출된 연속 리듬 구간이 부족합니다") 원인과 해결:

- **원인**: 기존 녹화기는 물리 스캔 코드(`KeyA` , `Digit1` …)가 있는 `keydown` 만 기록했으나, Android Gboard·iOS 소프트 키보드는 `code = ''` , `key = 'Unidentified'` (또는 `Process`)와 즉시 `keyup` 을 발생시켜 유효 전이가 `0개` 로 탈락했습니다.
- **해결** (`web/src/recorder.js`): ① `input` 이벤트 차분(한글 음절은 두벌식 자모 키로 분해: `안녕` → `ㅇㅏㄴㄴㅕㅇ`)으로 소프트 타건을 복원하고, ② 결측 홀드는 전이 양 끝 키별로 독립 대체(`88ms` , 원래 관측 마스크 보존으로 `Hold_Valid = 0`)하며, ③ 키 이벤트 없는 삭제는 `30ms` 조합 대기 후 소프트 백스페이스로 기록하고, ④ 자동완성·붙여넣기·OS 자동 반복은 합성 타건으로 분리해 리듬 전이에서 제외했습니다(터치 전용 기본 보정 임계값: 암호 `0.3534` , 1문장 기각 `0.6137`).
- **검증**: `web/scripts/parity_check.mjs` 가 Python 복제본과 JS 녹화기·전처리·점수 엔진 간 **`7,228` 개 수치 및 계약 항목 100% 일치**를 검증하고, `web/scripts/offline_smoke.mjs` (외부 통신 완전 차단 데스크톱·iPhone 한글 IME 스모크 **`33/33`**) 및 `web/scripts/adversarial_regression.mjs` (XSS·불량 스키마· `v4` 격리·실시간 2-브라우저 동기화 **`44/44`**)를 모두 통과했습니다.



5.6 사칭 차단 강화: 디그래프 합성 문자열·타이핑 스타일 대규모 시험

CMU 데이터에는 비밀번호가 `.tie5Roanl` 하나뿐이라, 사용자가 직접 정한 임의 비밀번호에서 사칭이 얼마나 통과하는지 정밀하게 측정하기 위해 실제 사람의 자유 타이핑 키 쌍 타이밍을 재조합한 **디그래프 합성 시뮬레이션**을 구축했습니다. 모든 보정 기준값은 `Validation` 인원(`10명`)에서만 정했고, 아래는 `Test` 인원(`18명`) 결과입니다.

시험 방법 및 표본 비중첩 분리 (`src/password_synth.py`)

- 미학습 피험자(`Validation 10명` , `Test 18명`)의 자유 타이핑에서 각 원시 디그래프 출현마다 고유 식별자(`occ_id`)를 부여하고, 한 시점 주변(`±2분`) 창에서 키 쌍 타이밍을 추출해 임의 문자열 입력을 재조합합니다. 특히 같은 날(`Session 1`) 평가 시 **등록(`enroll`)에 사용된 `occ_id` 집합과 본인 질의(`same`)에 사용되는 `occ_id` 집합을 100% 비중첩 분리 (`disjoint_occ_enforced = true`)**하여 동일 타건 재사용 편향을 원천 제거했습니다.
- 검증·시험 인원별로 서로 겹치지 않는 단어형 문자열(`6~10자`) 30종과 무작위 소문자열(`8자`) 30종, 총 `60` 종을 평가했습니다 (`Test 18명` 기준 **생성 입력 수**: 본인 `5,958` 회 / 사칭 `31,196` 회).

① 비밀번호: 9채널 + 17채널 듀얼 인코더 점수 결합 — 두 인코더의 템플릿 코사인 유사도(raw_9, raw_{17})를 검증 합성 사칭 분포로 각각 표준화해 합산합니다: $z = (raw_9 - \mu_9)/\sigma_9 + (raw_{17} - \mu_{17})/\sigma_{17}$. 판정 기준 τ 는 검증 본인 입력 하위 10% 지점이며, 웹 화면 점수는 $\text{sigmoid}(z - \tau)$ (기준 `50%`)로 표시됩니다.

시험 인원(18명) · 합성 문자열 60 종 (occ_id 비중첩)	**이전 (9채널 단독 AS-Norm)**	**듀얼 인코더 결합 (9ch + 17ch)**
키보드 `AUC` / `EER`	`0.842` / `22.8%`	**`0.874` / `20.4%`**
키보드 사칭 통과율 / 본인 거부율 (`6~10자` 공통 τ)	**`38.7%`** / `12.2%`	`41.9%` / **`7.7%`** *(→ §5.7 길이별 기준· 12자+ 에서 **19.1% / 12.2%** 로 해결)*
터치 `AUC` / `EER`	`0.739` / `32.5%`	**`0.770` / `30.0%`**
터치 사칭 통과율 / 본인 거부율 (`6~10자` 공통 τ)	`49.4%` / `18.9%`	`61.9%` / `8.8%` (모바일 참고용)

- 비중첩 분리 조건에서도 듀얼 인코더 결합은 단일 9채널 대비 순위 판별력(`AUC 0.842 → 0.874` , `EER 22.8% → 20.4%`)을 일관되게 높였습니다. 다만 `6~10자` 혼합 검증셋에서 하나의 공통 τ (`0.0081`)를 쓰면 문자열 길이에 따른 점수 이동 때문에 짧은 암호에서 동작점이 느슨해지며, 이 현상이 §5.7의 **길이별 보정 임계값 및 `12자 이상` 정책** 도입 근거가 되었습니다.
- 합성이 아닌 **실제 타건 문자열**(`Validation` 미병합 순수 `Test` , 등록 갱신 미적용, `6~10자` 공통 τ 기준)에서도 비교했습니다(`src/eval_password_calibration.py`):

실제 타건 데이터 (Test 전용: 사칭 통과율 / 본인 거부율)	**이전 (9ch AS-Norm)**	**듀얼 인코더 결합 (6~10자 공통 τ · AUC 비교)**
CMU `.tie5Roanl` 키보드 (`10명`)	**`17.1%`** / `14.9%` (`AUC 0.915`)	`37.0%` / **`3.3%`** (**`AUC 0.922`** , 웹 배포는 `10자` 전용 `τ=0.637` 적용)
반복 단어 키보드, 같은 날 / 다음 날 (`18명`)	**`25.5%` · `41.2%`** / `13.1%` · `4.3%` (`AUC 0.918·0.885`)	`27.1%` · `44.6%` / **`11.5%` · `0.0%`** (**`AUC 0.931·0.911`**)
KeyRecs 고정문 `vpwjkeurkb` 다음 날 키보드 (`18명` , `1,795/3,060` 건)	`31.8%` / `30.9%` (`AUC 0.755`)	**`21.3%`** / `35.7%` (**`AUC 0.808`** → 아래 자동 갱신으로 거부율 해결)
CMU 터치 (`10명`)	`17.0%` / `28.4%` (`AUC 0.858`)	`56.1%` / `5.2%` (**`AUC 0.873`** , 모바일 참고용)
반복 단어 터치, 같은 날 / 다음 날 (`18명`)	`15.6%` · `67.7%` / `18.0%` · `8.7%`	`21.9%` · `73.5%` / `6.6%` · `4.3%` (모바일 참고용)

② 고신뢰 등록 리듬 자동 갱신 — 처음 친 비밀번호가 손에 익으면 초기 4회 템플릿과 멀어져 본인 거부율이 상승합니다(위 KeyRecs 다음 날 `35.7%`). 검증셋에서 선택한 높은 여유 마진(**`z ≥ τ + 1.5`**)으로 승인된 동일 입력 방식(`keyboard` / `touch`) 타건만 템플릿에 추가(최근 `12개` 슬라이딩 유지, 샘플 계정 제외)하면 숙련 후 본인 거부가 거의 사라집니다.

KeyRecs 고정문 `vpwjkeurkb` (세션당 100회, 첫 4회 등록, Test 18명)	**이전 (AS-Norm)**	**결합 (갱신 없음)**	**결합 + 자동 갱신 (`z ≥ τ + 1.5` , 배포)**
키보드 본인 거부율(FRR): 당일 `5~14` / `51~100` / 익일 `1~10` / `91~100` 회	`2.2` / `21.0` / `20.6` / `27.4%`	`3.9` / `21.6` / `21.7` / `33.7%`	**`1.1` / `1.0` / `2.2` / `0.6%`**
키보드 사칭 통과율(FAR)	`30.3%`	**`22.2%`**	`25.4%`
터치 본인 거부율(FRR , 동일 구간 순서)	`2.2` / `23.1` / `20.0` / `40.6%`	`2.8` / `25.0` / `25.0` / `37.1%`	**`0.6` / `1.7` / `2.8` / `3.4%`**
터치 사칭 통과율(FAR)	`41.3%`	**`31.7%`**	`39.0%`

③ 타자연습: 등록 5문장 + 판별 2문장 오픈셋 프로토콜 — 등록을 3문장에서 `5문장` 으로, 블라인드 판별 입력을 1문장에서 `2문장` (두 임베딩 평균 + 행동 통계 합산)으로 확장했습니다. 검증셋에서 정답 인정률을 유지하며 외부인 오통과를 최소화하는 규칙을 탐색한 결과 **키보드 임계값 `0.84` (마진 `0.00`), 터치 임계값 `0.73` (마진 `0.00`)**이 선택되었고(웹 엔진은 `arena_margin > 0` 일 때 1·2위 근소 격차 `판별 보류` 로직을 함께 지원), 시험 인원 `18명` 에서 무작위 갤러리 300회 반복 추출로 검증했습니다.

시험 인원(18명) 무작위 갤러리 300회	**외부인 통과율: 이전(3+1) → 배포 (5+2)**	**타인 오식별률: 이전 → 배포**	**등록자 정답 인정률: 이전 → 배포**
키보드 `3명` 등록	`2.2%` → **`0.4%`**	`0.6%` → **`0.1%`**	`48.2%` → **`50.0%`**
키보드 `5명` 등록	`3.4%` → **`0.7%`**	`1.2%` → **`0.2%`**	`48.6%` → **`50.3%`**
키보드 `10명` 등록	`6.6%` → **`1.3%`**	`2.5%` → **`0.3%`**	`46.7%` → **`48.9%`**
터치 `3명` 등록	`10.6%` → **`4.5%`**	`3.6%` → **`1.3%`**	`53.4%` → `49.6%`
터치 `5명` 등록	`16.0%` → **`6.9%`**	`7.2%` → **`2.9%`**	`53.6%` → `50.6%`
터치 `10명` 등록	`28.8%` → **`13.3%`**	`14.3%` → **`6.1%`**	`49.3%` → `48.0%`

- **추가 연습 본인 확인 게이트**: 등록된 참가자 칩을 눌러 문장을 더 칠 때는 1문장 1:1 검증 임계값(검증 본인 거부율 10% 기준: 키보드 `0.6934` , 터치 `0.4850`)을 통과한 문장만 프로필에 반영되어, 타인 타건 혼입률을 키보드 **`2.85%`** (`본인 미반영 17.01%`), 터치 `11.61%` (`본인 미반영 19.32%`)로 억제합니다. 샘플 프로필은 수정 및 동명 등록이 모두 차단됩니다.

속도가 비슷한 사칭이 통과하는 구조적 이유 (`src/eval_typing_style.py` → `models/typing_style_analysis.json`)

- **속도·홀드 수준이 가까울수록 사칭 통과율이 높아집니다**: 사칭자와 대상자의 평균 키 간격 차이가 `5%` 미만이면 키보드 사칭 통과율이 `49.5%` , `35%` 이상이면 `35.8%` 이며(`6~10자` 기준; 이전 `45.9%` vs `32.5%`), 박자 모양은 둔 채 평균 간격만 대상자에게 맞추면 `44.2% → 54.6%` , 평균 홀드까지 맞추면 `75.4%` (터치는 간격 일치만으로 `65.3% → 76.8%`)까지 올라갑니다.
- **분산 분해(`유효 청크 통과 96명` , 원시 `99명` 중) 및 정보량 한계**: 자유 타이핑 키 간격 분산을 분해하면 키 배열 공통 `8.0%` , 개인 전체 속도 `19.7%` , **개인 고유 리듬(사람×키 쌍) `14.8%`** , 매회 흔들림 `57.5%` 입니다(홀드 시간은 키 배열 `4.4%` , 개인 수준 `35.5%` , 고유 리듬 `10.4%` , 흔들림 `49.7%`). 따라서 **각 키 구간이 독립 가우시안 분포를 따른다는 이상적 가정 하에서도**, 속도가 같은 두 사람을 고유 리듬만으로 가르는 판별기의 이론적 최소 EER은 간격 `9개` 에서 `36.2%` (홀드 `10개` 추가 시 `31.6%`), 문장 `1개` (약 `60개` 간격)에서 `17.4%` , 문장 `2개` (`120개` 간격)에서 `9.2%` 입니다. 판별 입력을 2문장으로 늘리고 신규 비밀번호를 `12자 이상` 으로 제한한 이유가 바로 이 신호대잡음비 한계 때문입니다.

5.7 비밀번호 길이와 대문자: 12자 이상 + 길이별 기준 + 대문자 습관

NIST SP 800-63B Rev. 4 권고(복잡도 강제 대신 길이 확대)가 키스트로크 생체 인증에도 유효한지 `occ_id` 비중첩 합성 방식으로 검증했습니다(`Validation 10명` 에서 길이별 `8·10·12·14·16·20자` 보정 파라미터 적합 → `Test 18명` , 길이별 무작위 문자

열 15종 평가, src/password_synth.py).

① 비밀번호 길이별 기준 보정 (Length-Aware Calibration) — 듀얼 결합 점수는 문자열이 길어질수록 평균이 상승하므로, 6~10자 공통 기준을 긴 암호에 그대로 쓰면 사칭 통과율이 오히려 증가합니다. 비밀번호 길이에 맞춰 $(\mu_9, \sigma_9, \mu_{17}, \sigma_{17}, \tau)$ 를 선형 보간해 적용한 결과는 다음과 같습니다.

시험 인원(18명) · 길이별 무작위 합성 문자열 15 종	키보드 AUC / EER	키보드: 6~10자 고정 기준 → 길이별 보정 기준 (FAR / FRR)	터치: 고정 기준 → 길이별 보정 기준 (FAR / FRR)
8자	0.882 / 20.2%	36.1% / 8.4% → 33.0% / 9.9%	56.6% / 10.3% → 56.7% / 10.2%
12자 (신규 등록 최소 길이)	**0.926 / 15.5%**	40.2% / 2.9% → **19.1% / 12.2%**	66.6% / 3.2% → 44.0% / 10.3%
16자	**0.942 / 13.8%**	49.3% / 0.6% → **14.2% / 13.4%**	78.4% / 0.0% → 38.2% / 10.3%
20자	**0.951 / 12.0%**	54.2% / 0.4% → **12.3% / 11.6%**	78.8% / 0.0% → 32.1% / 8.3%

- 웹 앱은 신규 비밀번호 계정에 대해 **최소 12자 이상 (password_min_length = 12)**만 등록을 허용하며, CMU 벤치마크 샘플 계정(isSample = true)만 예외적으로 원문 10자 를 유지합니다.

② 대문자 입력 습관 (Shift vs CapsLock) 결합 및 배포 동일 임계값 검증

- KeyRecs 자유 타이핑에서 대문자를 입력한 95명 을 분석한 결과, 개별 Shift 선행 시간은 매회 변동이 컸으나 **대문자를 Shift 로 치는지 CapsLock 으로 치는지**는 25명 이 주로 CapsLock을 사용(전체 대문자의 31.0%)하며 **다음 날 재현율이 98.95%** 에 달하는 강한 행동 지문이었습니다.
- 검증셋에서 가중치 3.0 과 기준 보정 tau_offset = +1.20 (+1.1996)을 선택하고, **실제 웹 배포 코드 (web/src/biometrics.js)와 정확히 동일한 판정 임계값(prm.tau + tau_offset = 1.1650 + 1.1996 = 2.3646)**으로 첫 글자 대문자 12자 키보드 비밀번호(Test 생성 입력 수: 본인 876 회 / 사칭 6,072 회)를 평가한 결과, **사칭 통과율이 18.21% → 11.00%** 로 감소했습니다(본인 거부율 10.96% → 16.67%). 모바일 소프트 키보드는 수정자 키 이벤트가 노출되지 않아 물리 키보드 입력에서만 반영합니다.

한계: 합성 문자열 시험은 3글자 이상 고차 문맥 연쇄와 숫자·특수문자 전환을 완전히 대변하지 못하며, 터치 수치는 모두 누름 시간 제거 시뮬레이션입니다. 또한 고신뢰 자동 갱신(z ≥ τ + 1.5)에서도 극단적으로 높은 점수를 얻은 사칭 입력은 템플릿에 혼입될 여지가 남아 있습니다(§7.2, §9.3).

---

6. 서비스 프로토타입

학습된 KeyPrintNet 인코더 2개(문장용 17채널 599 KB , 비밀번호용 9채널 592 KB ONNX)를 탑재하여 즉시 체험 가능한 **미니멀 MVP 웹 애플리케이션(KeyPrint)**을 구축하였습니다(web/ , 배포 URL: https://qwertypotter6873.github.io/keyprint-guard/).

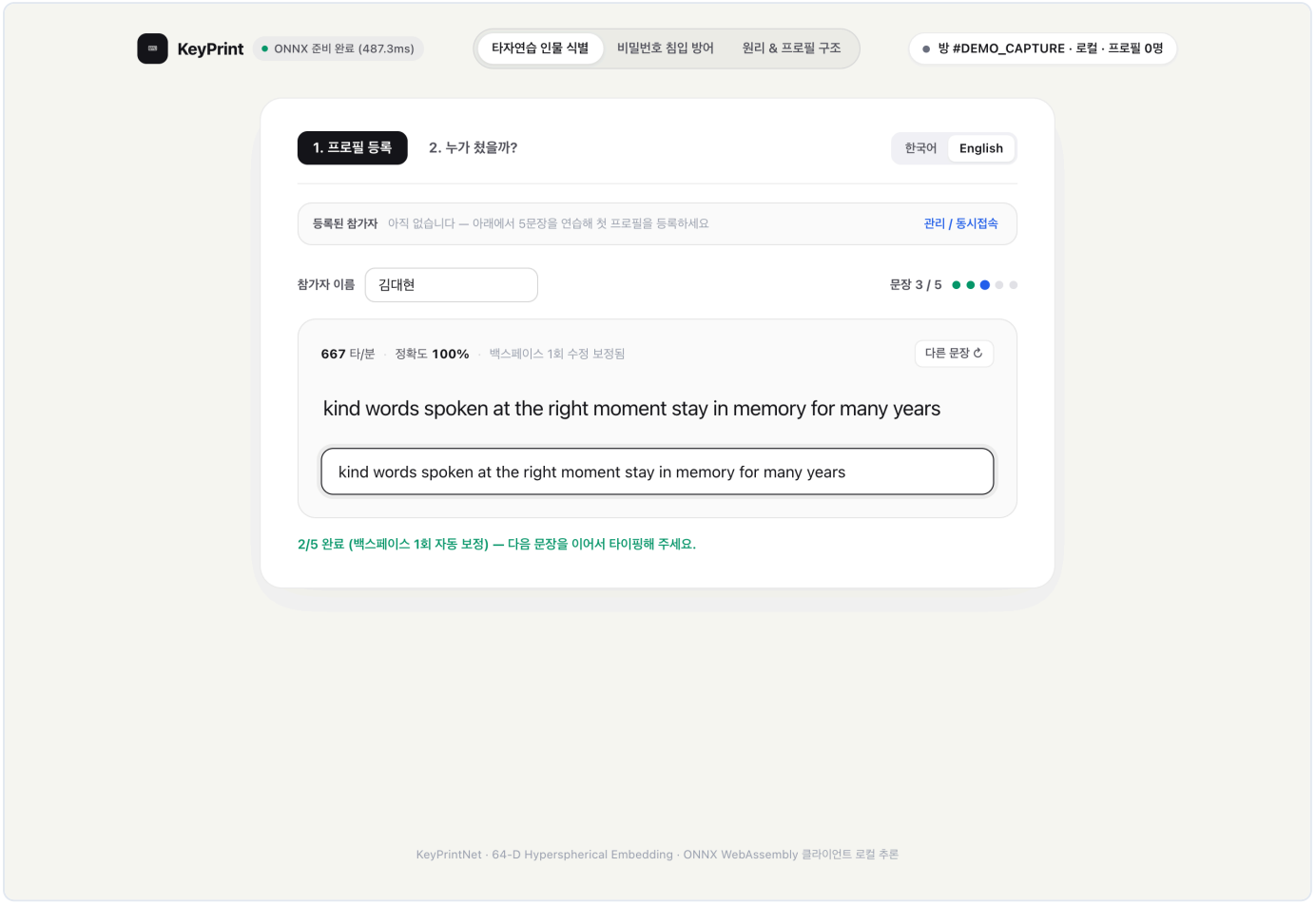
- **30초 체험 안내 (사용 방법)**:
  1. **타자연습 인물 식별(1:N)**: 상단 타자연습 인물 식별 탭 → 1. 프로필 등록 에서 참가자 이름을 입력하고 제시되는 한글/영문 문장 **5개**를 자연스럽게 타이핑합니다(Backspace 수정 허용). 이후 2. 누가 쳤을까? 탭에서 아무나 문장 **2개**를 이어서 치면 실시간 순위표와 예측 인물을 출력합니다.
  2. **비밀번호 침입 방어(1:1)**: 상단 비밀번호 침입 방어 탭 → + 새 암호 프로필 등록 에서 이름과 **12자 이상 시연용 비밀번호**(실제 개인 암호 입력 금지)를 4회 입력해 등록한 뒤, 로그인 방어 테스트 에서 본인 및 타인이 표시된 암호를 입력해 승인 (ACCESS GRANTED)·차단(INTRUSION BLOCKED)을 비교합니다(내장 샘플 4인으로 즉시 체험 가능).

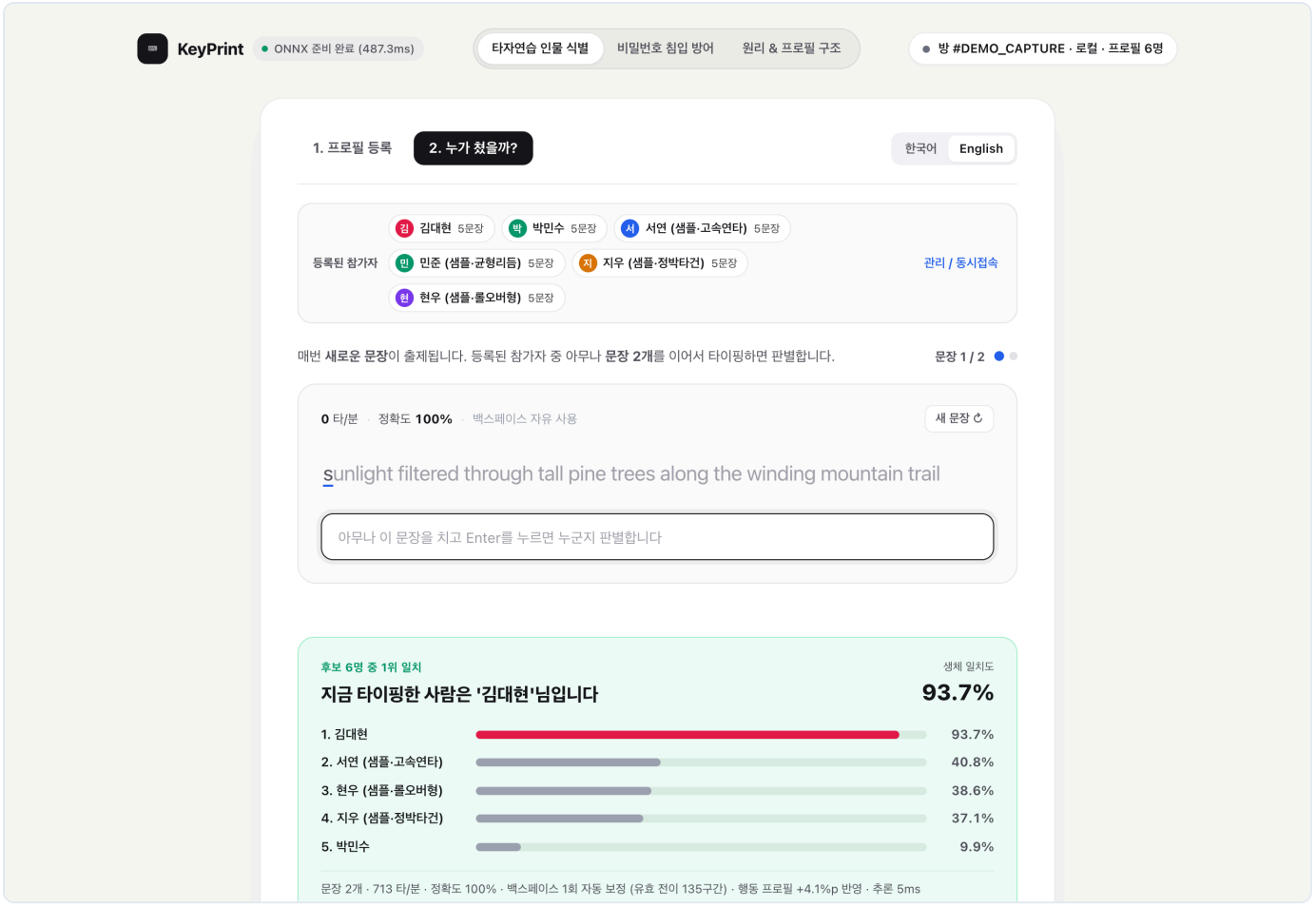
3. **원리 & 동시 접속**: `원리 & 프로필 구조` 탭에서 방금 판별된 타건의 피아노롤·64-D 바코드 수렴·2D PCA 지형도를 확인하고, 우상단 `방 #...` 버튼에서 초대 링크를 복사해 다른 PC·스마트폰과 실시간 교차 테스트를 수행합니다.

6.1 주요 기능 및 화면 워크플로우

① 타자연습 인물 자동 식별 (`1:N` — 다양한 문장 기반 · 백스페이스 자유 수정)

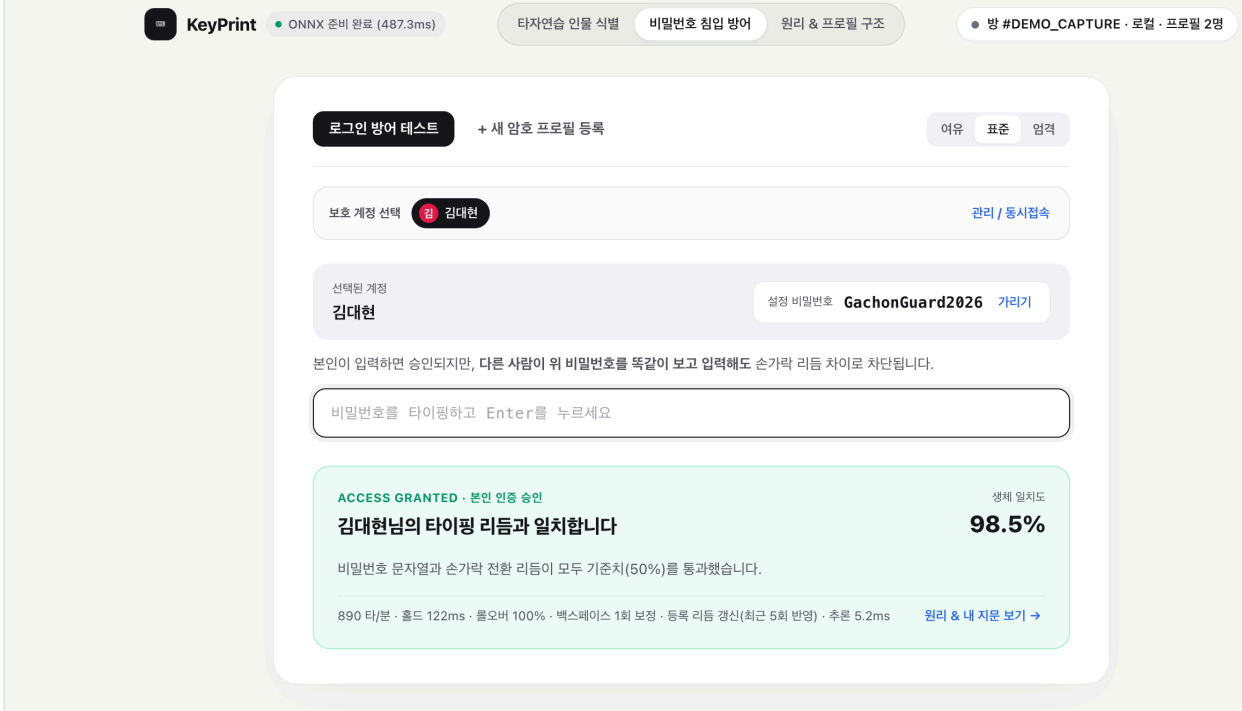
- **다양한 문장 연속 연습(`5문장`) 및 한글/영문 지원**: 같은 문장을 반복 입력하는 방식 대신 매회 서로 다른 한글·영문 문장(`각 24종 셔플 덱`)을 타이핑하며 실시간 타수(`타/분`), 정확도(`%`), 정오타 하이라이트를 확인합니다. 한글 IME 조합(`Process`) 중에도 두벌식 자모 스캔 코드 타이밍을 정밀 포착하며, 등록 화면에는 공유 방 공개 주의 안내가 상시 표시됩니다.
- **백스페이스 오타 수정 자동 보정(`Clean-Digraph Stack Reconstruction`)**: 입력 중 `Backspace` 를 눌러도 중단시키지 않고, 스택에서 삭제된 오타 전이만 격리한 뒤 전후 순수 연속 디그래프 구간을 이어 붙여(`Local_Accel` 접합점 `0` 리셋 및 전이별 홀드 개별 보존) 자연스러운 타이핑 그대로 `64-D` 생체 지문을 추출합니다.
- **모바일·행동 프로필 & 본인 확인 추가 연습**: 스마트폰 소프트 키보드(한글 조합 포함)로도 등록·판별이 가능하며, 행동 프로필 반영폭(`±%p`)과 입력 기기 전환 여부가 함께 표시됩니다. 등록된 참가자 칩을 눌러 추가 연습할 때는 1:1 본인 확인 임계값을 통과한 문장만 최근 `12개` 슬라이딩 템플릿에 반영됩니다(샘플 프로필 수정·동명 등록 차단).
- **블라인드 문장 판별(`누가 쳤을까?`) & 외부인 기각**: 미경험 문장 **2개**를 이어서 치면 참가자 전원의 일치도 순위 막대와 예측 인물을 출력하며, 1위 일치도가 오픈셋 임계값(`키보드 84.0%` , `터치 73.0%`) 미만이면 `등록되지 않은 새로운 사용자` 로 기각합니다(설정 마진 `arena_margin > 0` 인 경우 1·2위 근소 격차 시 `판별 보류` 표시).

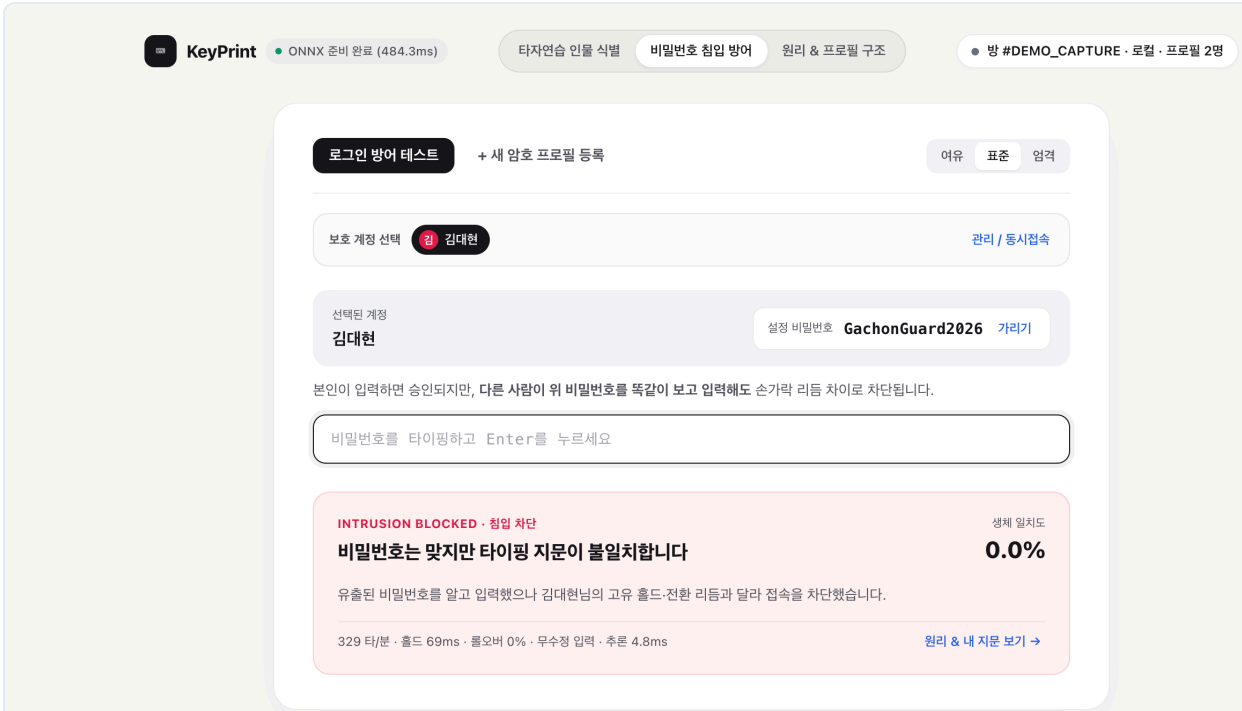




② 비밀번호 침입 방어 (`1:1` — 듀얼 인코더 · 동명 덮어쓰기 차단 · 실시간 교차 검증)

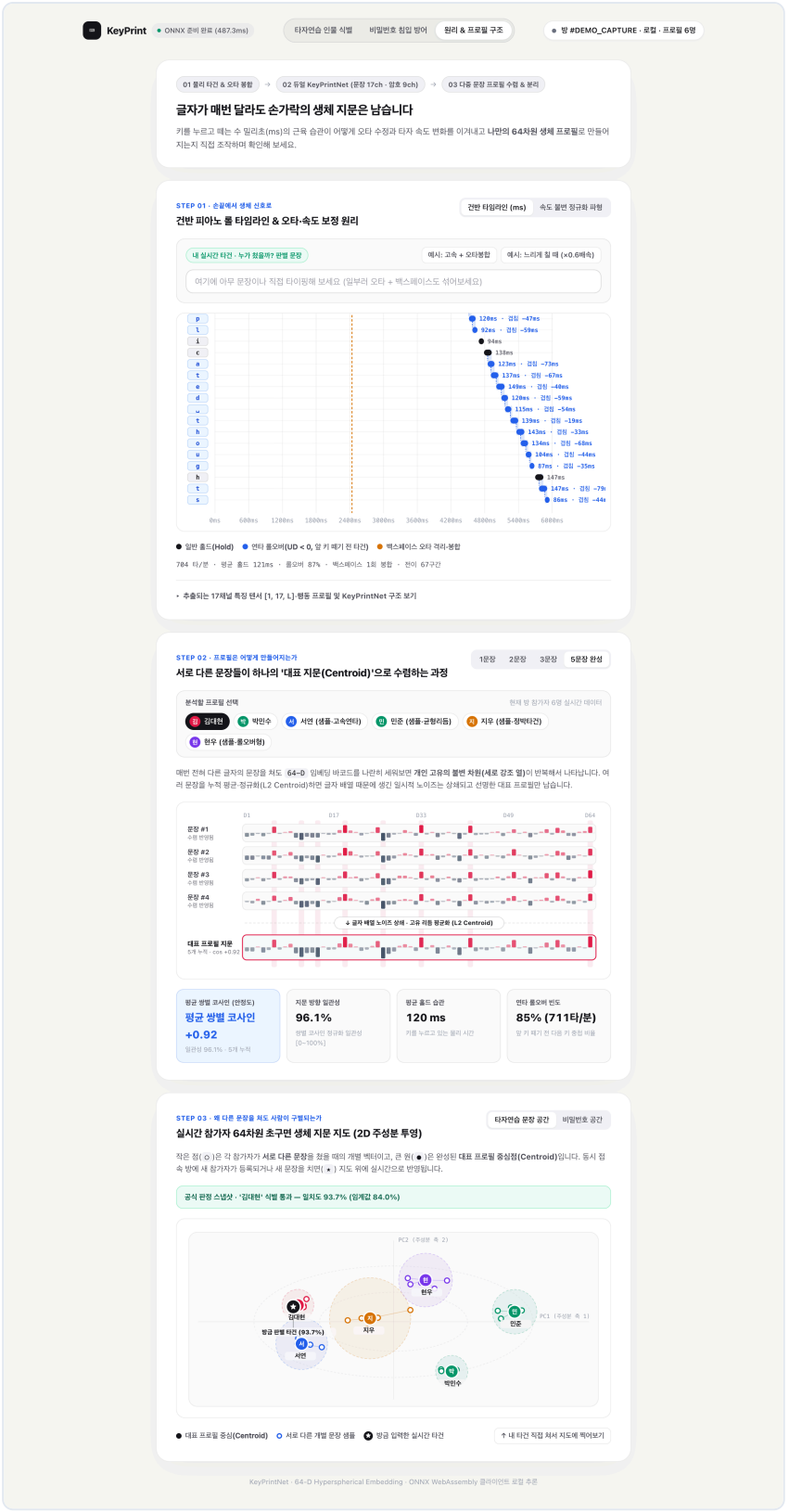
- **보호 계정 등록(`12자 이상`) 및 중복 덮어쓰기 방지**: 사용자별 이름과 시연용 비밀번호(`12자 이상`)로 4회 리듬을 등록합니다. 이미 비밀번호가 등록된 동일 이름이나 샘플 프로필 이름으로는 무인증 재등록(덮어쓰기)을 즉시 차단해 실수나 UI 우회를 방지합니다.
- **듀얼 인코더 점수 결합 + 고신뢰 자동 갱신**: 비밀번호 시도는 9채널·17채널 인코더 표준화 점수와 길이별 임계값, 키보드 대문자 습관(`Shift` / `CapsLock`)을 결합해 판정하며, 고신뢰($z \geq \tau + 1.5$) 승인 입력은 최근 12개 등록 샘플에 자동 반영되어 숙련 후 본인 거부를 막습니다(터치 입력은 `모바일 참고용` 표시).


KeyPrint
ONNX 준비 완료 (487.3ms)
타자연습 인물 식별
비밀번호 침입 방어
원리 & 프로필 구조
방 #DEMO_CAPTURE · 로컬 · 프로필 2명
로그인 방어 테스트
+ 새 암호 프로필 등록
여유
표준
엄격
보호 계정 선택
김
김대현
관리 / 동시접속
선택된 계정
김대현
설정 비밀번호
GachonGuard2026
가리기
본인이 입력하면 승인되지만, 다른 사람이 위 비밀번호를 똑같이 보고 입력해도 손가락 리듬 차이로 차단됩니다.
비밀번호를 타이핑하고 Enter를 누르세요
ACCESS GRANTED · 본인 인증 승인
생체 일치도
김대현님의 타이핑 리듬과 일치합니다
98.5%
비밀번호 문자열과 손가락 전환 리듬이 모두 기준치(50%)를 통과했습니다.
890 타/분 · 홀드 122ms · 롤오버 100% · 백스페이스 1회 보정 · 등록 리듬 갱신(최근 5회 반영) · 추론 5.2ms
원리 & 내 지문 보기 →
KeyPrintNet · 64-D Hyperspherical Embedding · ONNX WebAssembly 클라이언트 로컬 추론


KeyPrint
ONNX 준비 완료 (484.3ms)
타자연습 인물 식별
비밀번호 침입 방어
원리 & 프로필 구조
방 #DEMO_CAPTURE · 로컬 · 프로필 2명
로그인 방어 테스트
+ 새 암호 프로필 등록
여유
표준
엄격
보호 계정 선택
김
김대현
관리 / 동시접속
선택된 계정
김대현
설정 비밀번호
GachonGuard2026
가리기
본인이 입력하면 승인되지만, 다른 사람이 위 비밀번호를 똑같이 보고 입력해도 손가락 리듬 차이로 차단됩니다.
비밀번호를 타이핑하고 Enter를 누르세요
INTRUSION BLOCKED · 침입 차단
생체 일치도
비밀번호는 맞지만 타이핑 지문이 불일치합니다
0.0%
유출된 비밀번호를 알고 입력했으나 김대현님의 고유 홀드·전환 리듬과 달라 접속을 차단했습니다.
329 타/분 · 홀드 69ms · 롤오버 0% · 무수정 입력 · 추론 4.8ms
원리 & 내 지문 보기 →
KeyPrintNet · 64-D Hyperspherical Embedding · ONNX WebAssembly 클라이언트 로컬 추론

③ 원리 & 프로필 구조 인터랙티브 시각화 (`Signal-to-Profile XAI Explorer`)

- **Step 1 — 타건 신호 해부 & 속도 불변성 (`Piano-Roll & Tempo Invariance`)**: 키 눌림·해제 전 구간을 피아노롤 타임라인으로 펼쳐 보여주며, **연타 롤오버(`UD < 0`)**, **백스페이스 봉합(`✂ 가속도 0 리셋`)**, 속도 변화에도 유지되는 상대 박자(`Rel_DD = DD / 평균DD`)를 실시간 샌드박스와 호버 툴팁으로 확인합니다.
- **Step 2 — 64차원 프로필 수렴 및 평균 쌍별 코사인 안정도 (`64-D Barcode-to-Centroid`)**: 서로 다른 연습 문장(`#1~#5`)의 `64-D` 바코드가 초구면 중심점(`L2 Centroid`)으로 수렴하며 일시적 노이즈가 상쇄되는 과정과 샘플 간 **평균 쌍별 코사인 안정도(`Mean Pairwise Cosine` , `K < 2` 근거 부족 분리 처리)**를 탐색합니다.
- **Step 3 — 공식 판정 스냅샷 연동 2D 주성분 지형도 (`Live 2D PCA Constellation`)**: 방금 수행한 판정 시점(자동 갱신 직전 템플릿 기준)의 **공식 판정 스냅샷(`lastVerdict`)**과 실제 2D 주성분(`PCA`) 투영 좌표를 왜곡 없이 그대로 표시하여(방 이동·언어 변경·새 입력 시 자동 무효화), `1:N` 식별과 `1:1` 차단의 기하학적 근거를 투명하게 제시합니다.



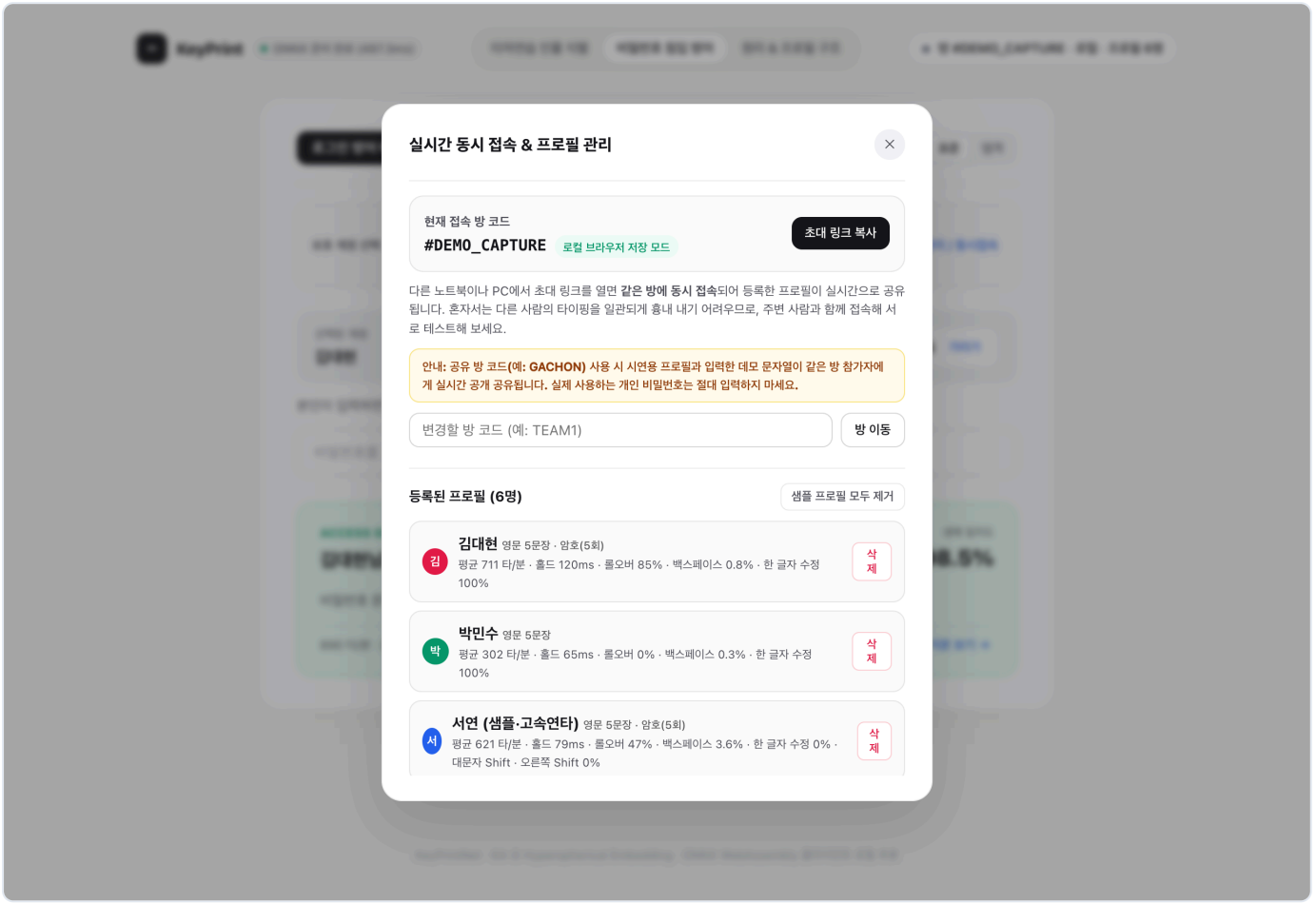
6.2 실시간 다중 기기 동시 접속(`Room Sync`) 및 입력 예외·보안 처리

여러 대의 노트북·스마트폰에서 동시에 접속해 프로필을 공유·교차 검증할 수 있는 **전 경로 버전·모델 해시 격리 동기화(`MQTT-over-WSS` + `BroadcastChannel` + `localStorage`)**와 엄격한 스키마·XSS 방어 처리를 구현했습니다(`web/src/roomSync.js` , `web/src/main.js`).

처리 코드 / 메커니즘	발생 조건	시스템 방어 및 자동 보정 동작
`BACKSPACE_STITCHED` *(자동 보정)*	입력 도중 `Backspace` / `Delete` 로 오타를 수정한 경우	오타 타건을 스택에서 제거하고 오염된 경계 전이만 제외한 뒤 전이별 홀드를 보존해 유효 디그래프 복원(`백스페이스 N회 자동 보정`)

처리 코드 / 메커니즘	발생 조건	시스템 방어 및 자동 보정 동작
`PAUSE_BRIDGED` *(자동 보정)*	문장 중간 `2.2초` 이상 멈춤 또는 이상치 홀드 발생	해당 비연속 경계에 `stitchFlags = true` (`Local_Accel = 0`)를 부여하고 전후 정상 타이핑 리듬 보존
`PASTE_BLOCKED` / `KEY_REPEAT_IGNORED`	붙여넣기(`Ctrl+V`) 시도 또는 키 길게 누름 (`e.repeat`)	붙여넣기 즉시 차단 안내; OS 자동 반복 글자는 합성 타건으로 분리해 리듬 전이에서 제외
`SOFT_KEYBOARD_INPUT` / `AUTOCORRECT`	모바일 소프트 키보드 입력 또는 자동완성 단어 치환	`input` 차분 복원 및 결측 홀드 개별 대체(`Hold_Valid = 0`); 일괄 치환 글자는 합성 타건 처리
`PASSWORD_MISMATCH`	[암호 모드] 입력 문자열이 설정 비밀번호와 불일치	문자열 불일치 안내 및 즉시 재입력 유도
`SHORT_OR_INCOMPLETE`	최소 길이(신규 암호 `12자` · 샘플 `10자` , 문장 `72%`) 또는 유효 전이(`8개`) 미달	신뢰 가능한 임베딩 추출을 위해 최소 글자 수 및 유효 연속 타건 구간 안내
`DUPLICATE_OR_SAMPLE_BLOCKED`	기존 암호 등록 이름 재등록 또는 샘플 프로필 이름 사용	무인증 암호 템플릿 덮어쓰기 및 샘플 프로필 변조 즉시 거절 안내
`SCHEMA_XSS_ISOLATION`	비정상 스키마·악성 문자열(`XSS`) 수신 또는 구버전/타 모델 해시 메시지	저장·렌더링 전 스키마 검증 폐기, 전 속성 HTML 이스케이프, `v4 · preprocessVersion · modelHash` 불일치 격리
`MODEL_LOADING_OR_OFFLINE_WS`	모델 초기화 중 `Enter` 입력 또는 `WebSocket` 차단 환경	초기화 대기 안내 표시 및 동기 `WebSocket` 예외 포착 후 로컬 브라우저 모드로 무중단 폴백

- 현행 프로토콜은 전 통신 채널(`MQTT` 토픽 `keyprint-agy-mvp-v4/...` , `localStorage` , `BroadcastChannel` , 삭제·초기화 이벤트 포함)에 `{ schemaVersion: 4, preprocessVersion: "kp-preproc-v4", modelHash: "ac38471759213dd1" }` 봉투를 적용해 구버전 프로필과의 공간 혼선을 차단합니다.



7. 배포

- **공개 서비스 데모 URL**: https://qwertypotter6873.github.io/keyprint-guard/
- **공개 소스코드 저장소**: https://github.com/qwertypotter6873/keyprint-guard

7.1 Zero-Server 브라우저 로컬 추론 아키텍처 채택 이유

본 시스템은 백엔드 추론 서버 대신 **PyTorch → ONNX(Opset 17, 문장 `599 KB` + 암호 `592 KB`) → ONNX Runtime Web(WASM SIMD) 정적 웹 배포(GitHub Pages)** 아키텍처를 채택했습니다.

1. **브라우저 내부에서 완결되는 신경망 추론**: 모든 전처리·임베딩 추출·점수 계산은 클라이언트 브라우저 WebAssembly 샌드박스 안에서 수행되며 원시 타건 타임스탬프는 외부 서버로 전송되지 않습니다. *(단, 선택 기능인 공유 방 동기화를 사용할 경우 방 참가자 간 교차 시연을 위해 프로필 템플릿과 시연용 암호 원문이 공개 MQTT 브로커로 전송되므로 화면에 실사용 암호 입력 금지 경고를 상시 표시합니다 — §7.2).*
2. **네트워크 왕복 없는 저지연 응답**: 서버 API 왕복(`50~300ms`)이나 무료 클라우드 콜드 스타트 없이, `Enter` 입력 직후 **브라우저 WASM 종단간 약 `1.5 ~ 5.2 ms`** (Python 로컬 순방향 추론 `0.4 ~ 0.9 ms`) 만에 임베딩 추출과 판정이 완료됩니다.
3. **CI/CD 무중단 자동 배포 (`.github/workflows/deploy.yml`)**: `main` 브랜치 푸시 시 GitHub Actions가 `npm ci`, ONNX WASM 바이너리 복사, Vite 프로덕션 빌드(`web/dist`) 및 GitHub Pages 배포를 자동 수행합니다. 단일 스레드 SIMD WASM 프로바이더와 동적 기준 경로(`new URL('./ort/', window.location.href)`)로 `SharedArrayBuffer` 헤더 제약 없이 모든 모던 브라우저에서 동작합니다.

7.2 보안 시스템으로 쓸 때 고려할 점 (현재는 시연용)

현재 웹 앱은 타이핑 생체 인식 원리와 다중 사용자 교차 검증을 체험하기 위한 **교육·시연용 프로토타입**입니다. 실제 운영 인증 시스템으로 전환하려면 아래 항목을 추가로 구현해야 합니다.

1. **반복 시도 제한 부재**: 시도 횟수 제한이 없으면 1회 통과 확률이 `p`일 때 `n`회 내 통과 확률은 `1 - (1 - p)^n`으로 커집니다. 12자 키보드 비밀번호(`p = 19.1%`, §5.7)는 독립 시도 가정 시 3회 내 약 `47%`, 10회 내 약 `88%`이며(대문자 습관 반영 `p = 11.0%` 기준 3회 내 약 `30%`), 실패 후 2회 연속 통과 요구 및 5회 실패 잠금 정책이 필요합니다.
2. **세밀한 일치도(%) 노출**: 시연 이해를 위해 매 시도 일치도(`%`)를 그대로 표시하므로 공격자가 점수를 보며 속도를 조절하는 언덕 오르기(hill-climbing)가 가능합니다. 실서비스에서는 실패 시 승인/차단만 반환해야 합니다.
3. **자동 갱신 오염 가능성**: 높은 여유 마진(`z ≥ τ + 1.5`)을 두었더라도 우연히 고득점으로 통과한 사칭 입력이 템플릿에 추가될 수 있으므로, 직전 실패 이력 존재 시 갱신을 동결하는 오염 방지 게이트가 필요합니다.
4. **클라이언트 측 판정의 한계**: 브라우저 내부 판정은 개발자 도구나 합성 이벤트 주입으로 우회할 수 있으므로 실서비스에서는 서버 측 검증 API로 이전해야 합니다.
5. **공개 무인증 MQTT 방 동기화 한계**: UI에서는 동일 이름 비밀번호 덮어쓰기와 비정상 스키마를 차단하지만, 공개 MQTT 브로커 자체는 클라이언트 소유권 인증을 수행하지 않으므로 방 코드를 아는 제3자가 브로커 패킷을 구독·발행할 수 있습니다(모델 해시는 임베딩 호환성 검사용이며 보안 인증 수단이 아님). 실서비스에서는 비밀번호 원문 대신 서버 측 해시 저장 및 인증된 WebSocket 채널을 사용해야 합니다.
6. **실패 사유 분리 표시에 따른 정보 노출**: 시연 화면은 `"비밀번호 문자열 불일치"`와 `"타이핑 지문 불일치"`를 구분해 보여주므로, 비밀번호 추측 상황에서는 `"로그인 실패"` 단일 메시지로 통합해야 추측 공격 성공률을 리듬 통과율(`×0.11 ~ ×0.19`)만큼 실질적으로 낮출 수 있습니다.

---

8. 실행 방법

8.1 저장소 구조 및 학습 가중치 위치

- **실행 환경**: `macOS / Linux / Windows`, `Python 3.10+` (`3.12` 검증, Apple `MPS` / NVIDIA `CUDA` / `CPU` 자동 감지), `Node.js 20+` (`22` 검증).

- **최종 모델 가중치 위치**: 문장용 17채널 `models/keyprint_net.pt` · `models/keyprint_net.onnx` (웹 배포본 `web/public/models/keyprint_net.onnx`, `599 KB`), 비밀번호용 9채널 `models/legacy/keyprint_net_9ch.pt` · `models/legacy/keyprint_net_9ch.onnx` (웹 배포본 `web/public/models/keyprint_net_password.onnx`, `592 KB`), 베이스라인 `models/vanilla_1dcnn.pt` · `models/vanilla_1dcnn_aug.pt`.

```
keyprint-guard/
├── README.md                                  # 프로젝트 포트폴리오 문서 (본 문서)
├── 포트폴리오_201935607_김대현.pdf              # 제출용 포트폴리오 PDF (A4 무잘림·외부 GitHub 링크)
├── submission.json                            # 자동 채점 메타데이터 명세
├── requirements.txt                           # Python 머신러닝 파이프라인 의존성 (scipy 포함)
├── data/
│   ├── raw/                                   # CMU & KeyRecs 원시 공개 데이터셋 CSV 3종
│   └── processed/scaler_and_metadata.json     # Train 적합 클리핑 경계·스케일러 및 분할 메타데이터
├── src/
│   ├── preprocessing.py                       # 원시 H/DD 보존 전처리, 무손실 파서, 비중첩 청크, 템포·터치 증강
│   ├── keyboard_layout.py                     # KeyboardEvent.code 물리 좌표·운지 표 및 두벌식 자모 매핑
│   ├── behavior_profile.py                    # 행동 프로필 16개 점수 특성 추출 + two-covariance LLR 적합
│   ├── capture_sim.py                         # 브라우저 녹화기의 Python 복제본 (전이별 홀드·봉합 100% 일치)
│   ├── feature_eval.py                        # 68자 웹 캡처 재현 평가, Validation 전용 β·인코더 선택
│   ├── password_synth.py                      # occ_id 비중첩 디그래프 합성, 듀얼 보정·길이별 기준·대문자 습관 평가
│   ├── eval_password_calibration.py           # 실제 타건 문자열 기준 비밀번호 보정 규칙 비교 (Test 전용)
│   ├── eval_typing_style.py                   # 속도·홀드 일치 실험, 96명 분산 분해, 독립 가우시안 정보량 한계
│   ├── models.py                              # ScaledManhattan, Vanilla1DCNN, KeyPrintNet,
SubCenterCosFace/SupCon
│   ├── train_and_evaluate.py                  # 전 모델 35-Epoch 직접 학습, 통제 비교, 1:1(인간/봇 분리) 및 1:N 평
가
│   ├── visualize_and_run.py                   # 단일 공식 파이프라인 실행, 8종 그래프 및 ONNX 2종·웹 설정 생성
│   ├── build_notebook.py                      # 저장 가중치 기반 비재학습 검증용 Jupyter Notebook 생성기
│   └── package_submission.py                  # 4단계 클린룸 검증 내장 제출용 ZIP 아카이브 패키징 스크립트
├── tests/
│   └── test_adversarial_pipeline.py           # 파서·봉합·원시 UD 보존·비중첩·빈 디렉터리 재현 단위/통합 테스트
├── notebooks/
│   └── keyprint_training_pipeline.ipynb       # 전 코드 셀 실행 완료(에러 0건)·공식 리포트 수치 일치 노트북
├── models/
│   ├── keyprint_net.onnx / .pt                # 문장용 17채널 KeyPrintNet (ONNX 599 KB)
│   ├── legacy/keyprint_net_9ch.onnx / .pt     # 비밀번호용 9채널 KeyPrintNet (ONNX 592 KB)
│   ├── vanilla_1dcnn.pt / vanilla_1dcnn_aug.pt # 기본 및 동일 증강 딥러닝 베이스라인 가중치
│   └── evaluation_report.json, password_calibration.json, typing_style_analysis.json
├── assets/
│   ├── figures/fig1..fig8_*.png               # 8종 데이터·학습·평가·오류 분석·기능 확장 그래프
│   └── screenshots/01..06_*.png               # 6종 최신 v2 웹 프로토타입 구동 스크린샷
└── web/                                       # Vite + ONNX Runtime Web 미니멀 MVP 애플리케이션
    ├── index.html, package.json, package-lock.json
    ├── src/ (recorder.js, features.js, validation.js, biometrics.js, visualizer.js, roomSync.js,
sentences.js, main.js, style.css)
    └── scripts/
        ├── parity_check.mjs                   # Python↔JS 7,228개 수치·봉합·스키마 계약 일치 검증
        ├── offline_smoke.mjs                  # 외부 통신 차단 데스크톱·모바일 브라우저 스모크 테스트 (33항목)
        ├── adversarial_regression.mjs         # XSS·불량 프로필·v4 격리·로컬 WS 브로커 2-브라우저 회귀 테스트 (44항목)
        ├── e2e_verify_and_capture.mjs         # 로컬 격리 방(#DEMO_CAPTURE) 프로토콜 검증 및 스크린샷 6종 촬영
        └── render_readme_pdf.mjs              # README → A4 포트폴리오 PDF 변환 및 링크·오버플로 자동 검수
```

8.2 머신러닝 파이프라인 재현 (전처리 · 직접 학습 · 평가 · ONNX 변환)

```
# 1. Python 가상환경 생성 및 패키지 설치 (Python 3.10+ 권장, CUDA / Apple MPS / CPU 자동 감지)
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 2. 전처리 + 전 모델(Vanilla 기본/증강, KeyPrintNet 9ch/17ch) 처음부터 학습 + 8종 그래프 & ONNX 내보내기
python src/visualize_and_run.py

# 3. 보조 평가 리포트 2종 생성 및 비재학습 검증 노트북 빌드·실행
python src/eval_password_calibration.py
python src/eval_typing_style.py
python src/build_notebook.py

# (선택) 저장된 가중치를 재사용하여 재학습 없이 평가·그래프·웹 설정만 동일하게 재생성
KEYPRINT_REUSE_WEIGHTS=1 python src/visualize_and_run.py

# 4. Python 적대적 회귀·재현 테스트 및 4단계 클린룸 제출 ZIP 생성·검증
python -m unittest discover -s tests -v
python src/package_submission.py
```

8.3 웹 프로토타입 로컬 실행 및 자동 E2E 검증

```
cd web
npm ci

# 개발 서버 실행 (predev 훅이 ONNX WASM 런타임을 public/ort/에 자동 복사)
npm run dev

# 스마트폰 실기기 테스트 (같은 Wi-Fi에서 표시되는 http://<PC IP>:5173 접속)
npm run dev -- --host

# Python↔JS 7,228개 항목 일치 + 프로덕션 빌드 + 오프라인 스모크(33) + 적대적 보안 회귀(44)
node scripts/parity_check.mjs
npm run build
node scripts/offline_smoke.mjs
node scripts/adversarial_regression.mjs

# 외부 브로커 차단 상태에서 로컬 격리 방(#DEMO_CAPTURE) E2E 검증 및 스크린샷 6종 재촬영
node scripts/e2e_verify_and_capture.mjs
```

---

9. Lessons Learned

9.1 PMI 회고 (Plus · Minus · Interesting)

- **Plus (좋았던 점)**:
  1. **사전학습 없는 순수 직접 학습(Scratch Metric Learning)과 엄격한 통제 비교로 두 과업 해결**: 외부 API나 사전학습 가중치 없이 도메인 생체 파생 피처(`Rel_H`, `Rel_DD`, `Rollover_Mag`)와 다중 스케일 팽창 SE-1DCNN + 1D-DCT 스펙트럼 풀링만으로 미학습 피험자 1:1 검증(`ROC-AUC = 0.9318`, `EER = 14.39%`, `Macro F1 = 0.8491`)과 웹 캡처 기준 18인 다른 날 문장 분류(`Top-1 = 81.30%`, `Top-3 = 96.45%`)를 각각 `600 KB` 미만의 경량 모델로 달성했습니다.
  2. **학습 파이프라인과 브라우저 서빙 간 100% 수학적 일치 확보**: 원시 `UD` 보존 클리핑 순서, 백스페이스 봉합 시 전이별 `(H_curr, H_next)` 보존, 행동 LLR, 듀얼 점수 및 대문자 습관 임계값까지 Python과 JS를 단일 수식으로 통일하고 **`7,228`개 자동 대조 항목** 및 PyTorch↔ONNX 최대 오차 **`3.58 × 10⁻⁷` 이하**로 검증하여 학습-서빙 괴리를 제거했습니다.

3. **판정 스냅샷과 직결된 신호→바코드→지형도 설명가능성(XAI)**: 단순 승인/차단 표시에 그치지 않고 실제 판정에 쓰인 타건의 **피아노롤 타임라인**, **`64-D` 바코드 중심점 수렴 및 쌍별 코사인 안정도**, 그리고 **적응 전 공식 판정 스냅샷이 연동된 무왜곡 2D PCA 초구면 지형도**를 제공해 생체 인식 근거를 투명하게 시각화했습니다.

- **Minus (아쉬웠던 점)**:
  1. **짧은 비밀번호에서의 본질적 정보량 한계**: 96명 타이밍 분산 분해 결과 전체 속도를 제외한 개인 고유 리듬 비중이 간격 `14.8%`·홀드 `10.4%`에 불과해, 전이 수가 적은 짧은 암호에서는 모델을 고도화해도 동일 속도 사칭 통과율을 낮추는 데 한계가 있었고 신규 등록 하한을 **`12자 이상`** (최소 전이 `L ≥ 5`, 샘플만 `10자`)으로 제한해야 했습니다.
  2. **물리 키보드 스위치 변경에 따른 잠재적 도메인 이동 한계**: 본 프로젝트의 학습·평가 데이터는 참가자별 고정 기기 환경에서 수집되었기 때문에, 기계식 스위치와 노트북 팬터그래프처럼 키 트래블·반발력이 다른 하드웨어로 기기를 바꿀 때 절대 홀드(`H_curr`) 분포가 이동할 수 있는 구조적 한계가 남아 있으며 교차 기기 실측 검증이 필요합니다.
  3. **모바일 터치 입력의 실기기 벤치마크 부재**: 소프트 키보드 등록과 한글 조합 문제는 해결했으나, 정량 터치 지표(다른 날 18인 `Top-1 = 55.49%`, `β = 0.30` 결합 시 `54.87%`)는 데스크톱 데이터에서 홀드를 제거한 시뮬레이션이며 검증셋 행동 가중치 이득이 테스트셋으로 이어지지 않아 실제 스마트폰 수집 데이터가 필요합니다.
  4. **단일 키 위치 인코더의 미학습 문자열 임계값 이동**: 문장 식별에서는 키 위치 7채널이 큰 폭의 성능 향상을 주었지만, 학습에 없던 짧은 비밀번호에서 단독 AS-Norm으로 쓰면 점수 분포가 이동해 `FAR`가 상승(`33.59% → 42.68%`)하는 양면성이 확인되어 9채널 인코더와의 듀얼 결합 보정이 추가로 필요했습니다.
- **Interesting (흥미로웠던 발견)**:
  1. **"똑같은 문장을 쳐도 드러나는 무의식적 롤오버 지문"**: 모든 참가자가 동일한 문구를 타이핑하더라도 어떤 참가자(`p003`)는 전이 구간의 **48%**에서 앞 키를 떼기 전 다음 키를 겹쳐 누르는(`UD < 0`) 반면, 다른 참가자(`p023`)는 **1%**만 겹쳐 누르는 극적인 신경근육 차이가 관찰되었습니다.
  2. **작가와 어휘가 완전히 바뀐 다른 날 소설에서도 유지되는 개인 리듬**: `KeyRecs` 실험에서 세션 1(Proust 소설)로만 등록하고 다른 날 세션 2(**Murakami 소설『해변의 카프카』**)의 전혀 다른 문장을 타이핑했을 때도 25자 청크 18명 중 **`Top-1 = 60.51%`, `Top-3 = 89.96%`** (17채널 기준; 9채널 `53.85% / 86.59%`)로 작성자를 찾아내어 타이핑 지문이 특정 단어 암기가 아닌 범용 운동 제어 특성임을 확인했습니다.
  3. **대문자 타이밍보다 훨씬 강력했던 `Shift` vs `CapsLock` 전략 지문**: 대문자 1개를 칠 때의 미세 Shift 타이밍은 매회 흔들림이 커서 이득이 없었던 반면, 대문자를 Shift로 치는지 CapsLock으로 치는지는 다음 날 재현율이 **`98.95%`**에 달해 첫 글자 대문자 12자 암호의 사칭 통과율을 `18.21% → 11.00%`로 낮추는 핵심 단서가 되었습니다.
  4. **원시 자유 타이핑 로그의 수정자(`Shift`) 음수 지연과 비이스케이프 따옴표 구조**: `KeyRecs-free-text.csv`에서 대문자 입력 시 인접 행의 음수·양수 `DD`(`-0.796s`, `+1.148s`) 합산값(`+0.352s`)이 브라우저의 글자 간 `DD`와 정확히 일치하고, 비이스케이프 큰따옴표(`"`) 타건 26행이 일반 CSV 파서에서 13개 다중행으로 병합되던 문제를 줄 단위 파서로 복구하며 데이터 정합성의 중요성을 확인했습니다.

9.2 핵심 문제 해결 사례 (`문제 → 원인 → 시도 → 결과`)

[사례 1] 세션 숙련도 상승(Speed Drift)으로 인한 초기 등록자 오차단(`FRR`) 급증 문제 해결

- **문제**: 최초 가입 시점(Session 1)에 5회만 등록한 뒤 암호 입력에 익숙해진 Session 2~8의 고속 타건을 검증하면 통계 베이스라인(`ScaledManhattan`)의 `Macro F1`이 **`0.6374`(`EER = 35.44%`, `FRR = 42.44%`)**까지 떨어지며 정당 사용자를 침입자로 오차단했습니다.
- **원인**: 세션이 거듭되며 전체 타자 속도가 빨라질 때 절대 타이밍(`H, DD, UD`)만 비교하면 전 구간 지연 감소를 타인 타건으로 오판하고, 초기 미숙련 머뭇거림이 고정 템플릿에 남기 때문이었습니다.
- **시도**: ① 시퀀스 내 평균으로 정규화한 **템포 불변 상대 리듬 채널(`Rel_H`, `Rel_DD`)** 및 롤오버 채널을 설계하고, ② 원시 `H, DD` 단계의 **탄성 템포 워핑 증강(`0.80×~1.25×`)**과 **AS-Norm 코호트 교정**을 결합했으며, ③ 웹 비밀번호 모드에는 검증셋에서 선택한 **고신뢰 슬라이딩 템플릿 자동 갱신(`z ≥ τ + 1.5`)**까지 추가했습니다.
- **결과**: Session 1 단일 등록 → Session 2~8 드리프트 벤치마크에서 `Macro F1`을 **`0.6374` → `0.6784`(`Vanilla1DCNN`) → `0.7773`(`KeyPrintNet` 17채널, `EER = 23.00%`, `FRR = 13.78%`; 9채널 `F1 = 0.7442`, `FRR = 22.22%`)**로 끌어올

렸고, KeyRecs 100회 반복 입력 시험에서도 익일 후반부(91~100회) 본인 거부율을 33.7% → 0.6% 로 해소했습니다.

[사례 2] 고정 암호(L=10)와 자유 타자연습 문장(L=24) 통합 학습 시 클래스 프로토타입 간섭 해결

- **문제**: 고정 암호(CMU L=10 , KeyRecs-fixed L=9)와 매번 철자가 바뀌는 자유 필사 문장(KeyRecs-free L=24)을 단일 인코더로 동시 학습할 때, 같은 참가자의 고정 문자열과 자유 문장을 하나의 각도 마진 프로토타입에 묶고 처음부터 큰 마진을 부과하면 위치 고조파 특징과 자유 철자 변동이 충돌해 수렴이 불안정해지는 문제가 있었습니다.
- **원인**: 고정 암호에서는 특정 상대 위치 τ 에 항상 같은 키 쌍이 놓이는 반면 자유 문장에서는 매 청크 다른 철자가 위치하므로, 두 도메인이 동일한 분류 중심점을 공유하면 상반된 그래디언트가 백본을 흔들기 때문이었습니다.
- **시도**: 합성곱·스펙트럼 풀링 백본 가중치는 세 코퍼스 간 100% 공유하되, 학습 전용 SubCenterCosFaceHead 의 **클래스 인덱스를 코퍼스별로 비중첩 분리(35 + 68 + 68 = 171 클래스, K = 2 서브센터)**하고 코사인 마진(s = 16.0 , m = 0 → 0.08)을 처음 20 에포크 동안 코사인 스케줄로 완만하게 램프업했습니다(src/train_and_evaluate.py).
- **결과**: 공식 35 에포크 학습에서 후반부 발산 없이 검증 동일오류율이 **9채널 최소 Val EER = 13.61% (Val AUC = 0.9382), 17채널 최소 Val EER = 14.17% (Val AUC = 0.9377 , 문장 검증 1:N 최고 79.74%)**로 매끄럽게 수렴하며 단일 아키텍처로 고정 암호와 자유 문장 두 과업을 동시에 달성했습니다.

[사례 3] 정적 웹 배포(GitHub Pages)의 WASM 모듈 경로 해석 및 한글 IME 타건 캡처 해결

- **문제**: Vite 프로덕션 빌드 후 정적 서버에서 /assets/ort/ort-wasm-simd-threaded.jsep.mjs (404) 오류로 ONNX 초기화가 중단되었고, 한글 타자연습 시 Chrome IME 조합 중 e.key 가 'Process' 로 반환되어 자모 타건이 누락되었습니다.
- **원인**: 번들 스크립트가 /assets/ 하위에 위치해 상대 경로 './ort/' 가 잘못 해석되었고, 한글 조합 중에는 논리 문자 확정 전 상태(Process)로 이벤트가 전달되기 때문이었습니다.
- **시도**: new URL('./ort/', window.location.href).href 동적 경로 해석과 predev / prebuild WASM 자동 동기화 스크립트를 적용하고, 물리 스캔 코드(Key[A-Z]) 기반 두벌식 자모 매핑 및 e.isComposing 상태 Enter 마이크로태스크 지연 처리를 구현했습니다.
- **결과**: 로컬 정적 빌드와 GitHub Pages 하위 경로 배포 모두에서 콘솔·페이지 에러 **0건** 으로 초기화 및 한글·영문 실시간 추론에 성공했습니다.

[사례 4] 타자연습 중 Backspace 오타 수정 폐기 문제 및 봉합 시 Hold 키 오매핑 결함 해결

- **문제**: 초기에는 Backspace 가 1회만 눌려도 문장 전체를 무효화했으나, 미학습 18인의 웹 동일 68자 자연 필사 캡처(1,206 건)를 분석한 결과 **80.3% (968 건)에서 최소 1회 이상의 Backspace 가 발생**해 대부분이 폐기되었습니다. 또한 전후 정상 구간을 이어 붙일 때 첫 전이 이후 h1 만 적재하면 끊긴 다음 구간 첫 키(C→D 의 C) 홀드에 이전 구간 끝 키(B) 홀드가 잘못 연결되는 미세 결함도 발견했습니다.
- **원인**: 오타 수정 경계의 단일 전이(DD)만 인지 지연으로 오염될 뿐 나머지 디그래프는 유효함에도 전체를 버렸고, 봉합 루프가 연속 체인(h0, h1, h2…)을 가정해 비연속 구간 재시작 시 새 시작 키 홀드를 갱신하지 않았기 때문이었습니다.
- **시도**: SmartKeystrokeRecorder 와 capture_sim.py 양쪽에서 오타 타건 pop 후 **채택된 각 전이마다 (hCurr[t], hNext[t]) 쌍을 명시적으로 보존하고 모든 비연속·이상치 건너뜀 지점에 stitchFlags[t] = true (Local_Accel = 0)를 기록하는 개선된 Clean-Digraph Stitching** 을 구현하고 손계산 반례 단위 테스트로 검증했습니다.
- **결과**: 공식 실행 절제 로그(troubleshooting_ablations.backspace_stitch_subset) 기준, 원래라면 폐기되었을 **Backspace 포함 캡처 968 건(80.3%)을 단 한 건도 버리지 않고 18인 분류 풀링 Top-1 = 83.88% / Top-3 = 97.83%** (당일 84.80% , 익일 83.07% ; 백스페이스 3회 이상 중증 수정 633 건에서도 Top-1 = 83.89% / Top-3 = 98.10%)를 달성하여 무오타 캡처(238 건, Top-1 = 81.51% / Top-3 = 97.90%)와 동등한 정확도를 입증했습니다.

[사례 5] 모바일에서 "추출된 연속 리듬 구간이 부족합니다"로 프로필 등록이 막히는 문제 해결

- **문제**: 스마트폰 소프트 키보드로 문장을 끝까지 입력해도 INSUFFICIENT_CLEAN_KEYS (유효 전이 8개 미만) 예외로 등록이 불가능했습니다.
- **원인**: 소프트 키보드는 물리 KeyboardEvent.code 가 비어 있고 keydown 직후 keyup 이 발생해 기존 물리 키 전용 녹화기에서 유효 전이가 0개 가 되었기 때문이었습니다.

- **시도**: `input` 이벤트 문자열 차분(한글 음절 두벌식 자모 분해)으로 소프트 타건을 복원하고, 결측 홀드 중앙값 대체(`88ms`) 및 `Hold_Valid = 0` 마스킹과 미니배치 `18%` 터치 모사 증강을 결합했습니다.
- **결과**: 오프라인 모바일 에뮬레이션에서 한글·영문 **5문장 등록 → 2문장 판별 → 12자 비밀번호 등록·인증** 전 과정이 무오류로 동작하며, 터치 시뮬레이션 18인 다른 날 `68자` 문장 분류 Top-1이 `42.04%` (9채널 단순 대체) → **`55.49%`** (17채널 터치 모사 학습; 행동 결합 웹 배포 `54.87%`)로 향상되었습니다.

[사례 6] 백스페이스를 누르지 않았는데 모바일·Safari 한글 조합이 오타 수정으로 오인되던 문제 해결

- **문제**: 백스페이스를 전혀 누르지 않고 한글 문장을 입력했음에도 `백스페이스 N회 자동 보정` 이 찍히고 피아노롤과 판별 점수가 훼손되는 현상이 발생했습니다.
- **원인**: iOS·삼성 한글 키보드는 글자 조합 단계마다 `삭제 → 입력` 이벤트 쌍을 연달아 발생시키고(`ㅇ → 아`), Mac Safari는 음절 확정 시 조합 글자를 지웠다가 다시 쓰며, 종성 연음 이동(`간 + ㅏ → 가나`)에서도 중간 길이 감소가 일어나 기존 즉시 삭제 감지 로직이 이를 가짜 백스페이스(`안녕하세요` 기준 `7회` , 유효 구간 `0/11`)로 오판했기 때문이었습니다.
- **시도**: ① 키보드 백스페이스 키 없이 발생한 삭제는 즉시 확정하지 않고 `30ms` 대기 창에서 직후 입력 이벤트와 병합해 자모 시퀀스의 순수 증감만 반영하도록 `web/src/recorder.js` 를 재설계하고, ② 조합 중인 마지막 음절은 오타 하이라이트에서 제외(`web/src/validation.js`)했으며, ③ 원리 탭 실시간 피아노롤 미리보기가 대기 버퍼를 부수지 않고 비파괴 조회(non-destructive peek)만 수행하도록 분리했습니다.
- **결과**: 동일한 iOS·Safari식 한글 조합 재현 테스트에서 가짜 백스페이스 **`0회`** , 유효 리듬 구간 **`11/11`** 을 복원하고 피아노롤에서도 `ㅇ ㅏ ㄴ ㄴ ㅕ ㅇ …` 자모 타임라인이 끊김 없이 렌더링됨을 자동 회귀 테스트로 검증했습니다.

9.3 향후 개선 계획 (데이터 · 모델 · 서비스 3차원 로드맵)

1. **[데이터 차원] 다기종 키보드 스위치 교차 실측 및 실제 스마트폰 터치 코퍼스 구축**: 동일 참가자가 기계식·팬터그래프·실제 스마트폰 소프트 키보드를 교차 사용하는 다기종 데이터를 직접 수집하여 스위치 변경에 따른 홀드 편차를 정량 보정하고, 시뮬레이션에 머물러 있는 모바일 터치 행동 가중치(`β`)와 센서 채널(`DeviceMotion` 미세 진동 등)을 실측 데이터로 재학습할 계획입니다.
2. **[모델 차원] 사용자 지정 비밀번호 맞춤형 경량 배경 코호트 생성기**: 현재는 코호트 편향을 피하기 위해 9채널·17채널 원시 유사도를 전역 사칭 분포로 표준화해 합산하고 있으나, 소형 디그래프 분포 요약표로부터 **사용자가 설정한 특정 비밀번호 문자열의 기대 사칭 점수 분포(`문자열 맞춤 μ, σ`)를 브라우저 로컬에서 즉시 추정**하면 짧은 암호와 터치 환경에서도 키 위치 정보를 더 정밀하게 정규화할 수 있습니다.
3. **[서비스·보안 차원] 오염 방지 적응 게이트 · 시도 제한 및 서버 측 검증 API 전환**: 현재 시연용 프로토타입에 미구현된 운영 보안 요구사항(§7.2)을 해결하기 위해, ① 직전 인증 실패 이력이 있을 때 템플릿 자동 갱신을 동결하는 **오염 방지 게이트(Anti-Poisoning Gate)**, ② 연속 실패 잠금 및 실패 시 정밀 점수(`%`) 비노출 처리, ③ 공개 무인증 MQTT 대신 **비밀번호 해시 저장 + 서버 측 ONNX 검증 API**를 갖춘 LMS·사내 SSO 인증 플러그인으로 고도화할 계획입니다.